

TRƯỜNG ĐẠI HỌC THĂNG LONG  
BỘ MÔN TIN HỌC



# BÀI TẬP LỚN

## AN TOÀN THÔNG TIN

**Đề tài: Nghiên Cứu Về Các Kỹ Thuật  
Phân Tích Tĩnh Trong Phân Tích Malware**

**Giáo viên hướng dẫn: Nguyễn Hữu Tiến**

**Sinh viên thực hiện: A42718 Lê Thảo Quyên**

**Hà Nội – 2024**

## **PHẦN 1. TỔNG QUAN VỀ MALWARE**

### **1.1. Tổng quan về Malware**

#### **1.1.1. Định nghĩa**

Malware là từ viết tắt của “Malicious software”, có nghĩa là phần mềm độc hại. Malware là một loại phần mềm được tạo ra và chèn vào hệ thống một cách bí mật với mục đích thâm nhập, phá hoại hệ thống hoặc lấy cắp thông tin, làm gián đoạn, tổn hại tới tính bí mật, tính toàn vẹn và tính sẵn sàng của máy tính nạn nhân.

#### **1.1.2. Phân loại**

Malware là thuật ngữ gọi chung cho các mối nguy hại, có thể phân loại như sau:

- Theo mục tiêu:
  - + Malware nhắm vào cá nhân
  - + Malware nhắm vào doanh nghiệp, tổ chức
  - + Malware nhắm vào cơ sở hạ tầng
- Theo tính năng:
  - + Malware gây hại, phá hủy
  - + Malware đánh cắp thông tin
  - + Malware kiểm soát hệ thống
  - + Malware hiển thị quảng cáo
- Theo phương thức lây lan:
  - + Malware lây lan thông qua email
  - + Malware lây lan thông qua mạng
  - + Malware lây lan thông qua tải về
- Theo kỹ thuật thực thi:
  - + Malware có mã độc
  - + Malware không có mã độc
  - + Malware sử dụng kỹ thuật khai thác lỗ hổng
  - + Malware sử dụng kỹ thuật lừa đảo

Các loại Malware phổ biến:

- Virus: gắn vào một chương trình khác và khi được thực thi, thường là do người dùng vô tình, nó sẽ tự sao chép bằng cách sửa đổi các chương trình máy tính và lây nhiễm chúng bằng các đoạn mã của chính nó.
- Worm: tương tự như virus, tuy nhiên điểm khác biệt lớn nhất là nó có thể tự lây lan trên các hệ thống, trong khi virus sẽ cần một số các hành động từ người dùng để bắt đầu lây nhiễm.
- Trojan: thường “giấu mình” trong những phần mềm hữu ích đối với người dùng, lôi kéo người dùng cài đặt chúng. Khi xâm nhập vào hệ thống, những kẻ tấn công sẽ truy cập trái phép vào máy tính, từ đó Trojan có thể được sử dụng để đánh cắp thông tin tài chính hoặc cài đặt các dạng phần mềm độc hại khác.
- Spyware: bí mật quan sát các hoạt động của người dùng máy tính mà không được sự cho phép, sau đó gửi những thông tin quan trọng cho tin tặc.
- Keylogger: bí mật ghi lại các phím đã được nhấn bằng bàn phím rồi gửi tới hacker. Keylogger có thể ghi lại nội dung của email, văn bản, tài khoản và mật khẩu người dùng, thậm chí cả chụp ảnh màn hình máy tính nạn nhân.
- Adware: dùng để thu thập thông tin cá nhân lưu trữ trên máy tính rồi hiển thị nhiều loại quảng cáo trên máy tính đã bị lây nhiễm. Adware có thể điều hướng trình duyệt web của bạn đến các trang web không an toàn, đồng thời nó còn có thể chứa cả Trojan và Spyware.
- Rootkit: cung cấp cho kẻ tấn công các đặc quyền của quản trị viên trên hệ thống bị nhiễm, hay còn được gọi là quyền truy cập "root".
- Ransomware: ngăn bạn truy cập vào thiết bị và mã hóa dữ liệu, sau đó buộc bạn phải trả tiền chuộc để lấy lại chúng. Ransomware được xem là vũ khí của tội phạm mạng vì nó thường dùng các phương thức thanh toán nhanh chóng bằng tiền điện tử.

## **1.2. Cơ chế hoạt động của Malware**

Cơ chế hoạt động của Malware bao gồm các bước chính sau đây:

- Lây nhiễm (Infection):

- + Malware xâm nhập hệ thống thông qua các phương thức như email, trang web độc hại, thiết bị lưu trữ di động, hoặc lợi dụng các lỗ hổng bảo mật.
- + Sau khi vào hệ thống, Malware sẽ chèn mã độc vào các tập tin, chương trình hoặc bộ nhớ.
- Ẩn mình (Evasion):
  - + Sử dụng kỹ thuật như mã hóa, làm rối mã nguồn (obfuscation), đóng gói (packing) để tránh bị phát hiện bởi phần mềm bảo mật.
  - + Rootkit có thể cài đặt sâu vào hệ điều hành để ẩn dấu sự hiện diện của mình.
- Thực thi (Execution):
  - + Malware chờ đợi một điều kiện nhất định hoặc sự kiện kích hoạt mã độc, như mở tài liệu văn phòng có macro.
  - + Tự động thiết lập để khởi động cùng hệ điều hành.
- Điều khiển và Cập nhật (Command and Control):
  - + Kết nối tới máy chủ điều khiển từ xa để nhận lệnh, thường thông qua các giao thức như HTTP, HTTPS, IRC, hoặc P2P.
  - + Tự động cập nhật mã độc để thêm tính năng mới hoặc khắc phục lỗ hổng.
- Thực hiện Hành vi Độc hại (Malicious Actions):
  - + Tự nhân bản và lây lan sang các hệ thống khác (virus, worm).
  - + Chiếm quyền kiểm soát hệ thống để truy cập từ xa, thực thi lệnh (trojan, rootkit).
  - + Phá hoại, mã hóa dữ liệu (ransomware).
  - + Thu thập thông tin cá nhân, tài chính hoặc nhạy cảm (spyware).

### **1.3. Cách phòng tránh Malware**

- Cập nhật phần mềm và hệ điều hành:
  - + Bật tính năng cập nhật tự động, thường xuyên kiểm tra, cài đặt các bản vá bảo mật mới nhất cho hệ điều hành, trình duyệt và các phần mềm khác.
  - + Sử dụng phần mềm có nguồn gốc rõ ràng và được cung cấp từ các nhà phát triển uy tín.

- Quản lý và giám sát an ninh hệ thống
  - + Giám sát các hoạt động, giao dịch và sự kiện bất thường trên hệ thống.
  - + Cảnh giác với các web có domain kết thúc bằng tập hợp các chữ cái riêng lẻ, có đuôi không giống như bình thường (.com, .vn, .org, ... ).
  - + Cấu hình chính sách và quyền truy cập phù hợp để hạn chế các hoạt động nguy hiểm.
- Sử dụng phần mềm diệt virus và phần mềm bảo mật
  - + Cài đặt và duy trì hoạt động của phần mềm diệt virus, phần mềm ngăn chặn mã độc uy tín.
  - + Thường xuyên quét và dọn sạch các mối đe dọa được phát hiện.
- Sử dụng công nghệ mới như AI, học máy để phát hiện và ngăn chặn mã độc:
  - + Các giải pháp bảo mật dựa trên AI có thể phát hiện và ngăn chặn các mối đe dọa mới, phức tạp một cách hiệu quả.
  - + Kết hợp nhiều giải pháp bảo mật khác nhau để tăng cường khả năng phòng vệ.
- Tăng cường ý thức bảo mật cho người dùng
  - + Sao lưu dữ liệu quan trọng một cách thường xuyên.
  - + Tránh nhấp vào các quảng cáo pop-up khi bạn lướt web.
  - + Không tải các phần mềm, ứng dụng ở trên các website không đáng tin cậy.
  - + Không nhấp vào các liên kết lạ, các liên kết không xác định ở trong email hay văn bản và tin nhắn.

## **PHẦN 2. CÁC KỸ THUẬT PHÂN TÍCH TÍNH TRONG PHÂN TÍCH MALWARE**

### **2.1. Mục tiêu phân tích Malware**

Mục tiêu phân tích Malware chủ yếu để cung cấp các thông tin cần thiết để xây dựng hệ thống bảo vệ và ngăn ngừa các nguy cơ do mã độc gây ra, bao gồm xác định chính xác những gì xảy ra khi thực thi mã độc, như các tập tin nào được mã độc sinh ra, các kết nối mạng bên ngoài được mã độc thực hiện, những thay đổi về registry liên quan đến mã độc, các thư viện và vùng nhớ liên quan đến hoạt động của mã độc,... Việc phát hiện ra hành vi mã độc sẽ giúp tạo ra các dấu hiệu nhận dạng mã độc để áp dụng vào hệ thống lọc bảo vệ mạng như dấu hiệu về host-base và dấu hiệu nhận dạng về mạng.

- Dấu hiệu nhận dạng host-base: được dùng để phát hiện mã độc trên máy tính nạn nhân. Những dấu hiệu này bao gồm các tập tin được tạo ra hoặc thay đổi bởi mã độc hoặc là những thay đổi trong registry. Không giống với cách nhận biết trên các trình diệt virus, dấu hiệu host-base chủ yếu tập trung vào những gì mã độc làm trên hệ thống bị nhiễm, điều này sẽ hiệu quả hơn khi mã độc tự động thay đổi hình thái hoặc tự xóa khỏi đĩa cứng.
- Dấu hiệu nhận dạng về mạng: được dùng để phát hiện mã độc bằng cách giám sát lưu lượng mạng trong hệ thống. Dấu hiệu nhận dạng về mạng có thể được tạo mà không cần quá trình phân tích mã độc, tuy nhiên dấu hiệu nhận dạng mạng sẽ hiệu quả hơn nếu được tạo với trợ giúp của quá trình phân tích mã độc.

Sau khi thu tập dấu hiệu nhận dạng, mục tiêu cuối cùng đó là tìm hiểu chính xác mã độc hoạt động thế nào, tức là làm rõ mục đích và khả năng của chương trình mã độc. Phân tích Malware là một bước quan trọng để có thể ngăn chặn và tiêu diệt hoàn toàn mã độc ra khỏi máy tính và hệ thống mạng, khôi phục lại hiện trạng ban đầu và truy tìm nguồn gốc tấn công.

### **2.2. Định nghĩa phân tích tính trong phân tích Malware**

Phân tích tính mô tả việc phân tích mã, cấu trúc của một phần mềm mà không cần thực thi chúng nhằm mục đích kiểm tra xem tập tin, phần mềm có phải mã độc hay không, và cố gắng xác định hành vi của mã độc.

Phân tích tính được phân loại thành phân tích cơ bản và phân tích nâng cao.

### **2.2.1. Phân tích tĩnh cơ bản**

Phương pháp phân tích tĩnh cơ bản bao gồm việc kiểm tra các tập tin thực thi. Phân tích tĩnh cơ bản có thể xác định một tập tin là độc hại, cung cấp các thông tin chức năng của tập tin độc hại đó, và đôi khi cung cấp thông tin cho phép chúng ta tạo ra các chữ ký mạng đơn giản. Phân tích tĩnh cơ bản thì đơn giản và nhanh chóng, tuy nhiên nó sẽ không hiệu quả khi gặp phải các phần mềm độc hại tinh vi hơn.

### **2.2.2. Phân tích tĩnh nâng cao**

Phương pháp phân tích tĩnh nâng cao bao gồm kỹ thuật dịch ngược (Reverse Engineering) nội dung bên trong của Malware bằng cách đọc tập tin thực thi vào một bộ phân tách và xem xét các chỉ thị của chương trình để phát hiện các chương trình nghi ngờ. Các chỉ thị được thực thi bởi CPU, phân tích tĩnh nâng cao sẽ cho biết chương trình nào bị nghi ngờ. Tuy nhiên, phân tích tĩnh nâng cao đòi hỏi kiến thức chuyên môn về lập trình, cấu trúc mã lệnh, và các khái niệm về hệ điều hành Windows.

## **2.3. Các kỹ thuật phân tích tĩnh trong phân tích Malware**

### **2.3.1. Quét virus**

Bước đầu tiên khi phân tích tĩnh đó là sử dụng các chương trình, phần mềm chống virus (anti-virus) để quét tệp chúng ta nghi ngờ.

Các công cụ, phần mềm chống virus có khả năng phát hiện mã độc dựa vào các cơ sở dữ liệu, các học thuật AI để phát hiện các đoạn mã độc hại trong tệp. Từ đó chúng ta dễ dàng phát hiện và kết luận được tập tin đáng ngờ.

Mỗi một chương trình chống virus có một cơ sở dữ liệu riêng, nên để tối ưu, chúng ta có thể kiểm tra bằng cách quét nhiều phần mềm chống virus như là Kaspersky, AVG, AVAST, Bitdefender, .... Hoặc đơn giản là sử dụng công cụ được triển khai dưới dạng web đó là VirusTotal. VirusTotal cho phép tải lên tệp và tự động quét bởi nhiều phần mềm anti-virus. Ngoài ra còn cung cấp thêm nhiều thông tin hữu ích khác.

Ưu điểm của việc sử dụng các antivirus sẽ giúp chúng ta nhanh chóng xác định được mẫu tệp nghi ngờ đã từng bị phát hiện hay chưa. Trong trường hợp không phát hiện mã độc qua các công cụ trên sẽ xảy ra hai khả năng:

- Khả năng đầu tiên là mẫu bạn đang phân tích thực sự là 1 tệp thông thường.

- Khả năng thứ hai, mẫu bạn đang phân tích thực sự là một mã độc “KHỦNG” vì nó là một mẫu mới hoặc nó đã qua mặt được các antivirus và hệ thống phân tích online.

Nhược điểm của phương pháp này là các phần mềm độc hại có thể thay đổi, làm rối hoặc che dấu các đoạn mã để vượt qua được các phần mềm diệt virus.

### **2.3.2. Kiểm tra mã băm (Hash)**

Hash đề cập đến việc chuyển đổi dữ liệu thành một hàm băm giá trị (giá trị băm) thông qua hàm cụ thể và quá trình này được gọi là "băm". Hàm băm thường được sử dụng trong nhiều ứng dụng khác như cách kiểm tra toàn bộ dữ liệu, lưu trữ mật khẩu an toàn, xác thực dữ liệu và trong các thuật toán tìm kiếm và sắp xếp dữ liệu. Các hàm băm phổ biến bao gồm MD5, SHA-1, SHA-256 và SHA-512.

Hàm băm mật mã là công cụ quan trọng trong lĩnh vực mã hóa học và an ninh thông tin. Mục tiêu chính của mã hóa hàm băm là tạo ra loại hàm băm duy nhất có giá trị cao nhất từ đầu vào dữ liệu. Mã băm được sinh ra từ những thuật toán, mỗi tệp tương ứng được xác định bởi 1 mã băm nên chúng ta cũng có thể sử dụng mã băm như một thông tin để tìm kiếm và xác định mã độc. Trong trường hợp ta không muốn tải mẫu của mình lên một hệ thống online hoặc đưa mẫu cho những nhà phân tích khác, ta có thể sử dụng phương pháp này.

PowerShell với công cụ Capa có thể giúp chúng ta tính toán mã băm phổ biến như: MD5, SHA... Ngoài ra, còn các công cụ miễn phí khác ta có thể sử dụng như: md5deep, WinMD5, CFF Explorer... Với mã băm thu được, ta có thể sử dụng cho nhiều mục đích khác nhau:

- Gán nhãn, đánh dấu một biến thể mã độc, trạng thái file sạch...
- Chia sẻ cho các chuyên gia, người dùng khác để họ hỗ trợ phát hiện hoặc đề phòng mã độc.
- Tải lên các trang web online như VirusTotal để xác định xem có loại mã độc nào tương tự hay chưa.

### **2.3.3. Phân tích chuỗi (String)**

Một chương trình thông thường sẽ bao gồm rất nhiều chuỗi (string), sử dụng để thực hiện in các thông báo, cảnh báo, hiển thị thông tin, cũng có thể là địa chỉ IP hoặc tên miền máy chủ mà phần mềm đó kết nối tới, có thể chứa các địa chỉ email, thông tin đăng nhập, mật khẩu, tài khoản mặc định...



Phân tích chuỗi có thể giúp tìm hiểu về chức năng của mã độc thông qua các chuỗi văn bản có trong tập tin thực thi. Lưu ý rằng chuỗi có thể được lưu dưới dạng ASCII và Unicode. Các phần mềm chuẩn thường sẽ có nhiều chuỗi, còn các phần mềm độc hại thường che dấu các chuỗi đáng ngờ nên chúng thường có ít chuỗi hơn. Các công cụ có thể giúp ích như:

- strings (Linux): công cụ strings có thể phân tích ASCII và Unicode đồng thời. Option *-n* giúp ta chỉ định độ dài chuỗi tối thiểu cần phân tích (mặc định là 3) và *-o* sẽ xuất ra decimal offset của chuỗi đó.
- BinText: tương tự strings nhưng với độ dài chuỗi tối thiểu cần phân tích mặc định là 5.
- Sysinternals Strings (Windows)

Với nhiều chương trình, có những chuỗi có thông tin ý nghĩa sẽ xuất hiện, hỗ trợ chúng ta hiểu về file cần phân tích rõ hơn. Tuy nhiên, hầu hết các mã độc ngày nay đều sử dụng các kỹ thuật encrypt, obfuscate, pack, archive... Các chuỗi có ý nghĩa như địa chỉ IP, tên miền thông thường phải đi qua các hàm giải mã mới lấy được, do đó việc xem các chuỗi không phải lúc nào cũng cho những kết quả như ý muốn.

#### **2.3.4. Kiểm tra trình đóng gói (packer)**

Các hacker viết mã độc thường sử dụng kỹ thuật packed (đóng gói) và obfuscated (làm rối) để làm cho mã độc khó bị phân tích hơn. Một chương trình được đóng gói là chương trình đã được nén lại và không thể phân tích tĩnh được. Một chương trình được làm rối là một chương trình mà mã nguồn của nó được xáo trộn, làm cho rối rắm, gây khó khăn trong việc phân tích tĩnh. Cả hai kỹ thuật này đều gây ra trở ngại lớn cho việc phân tích tĩnh một mã độc.

#### **Làm sao để có thể phát hiện một chương trình đã bị packed hoặc obfuscated?**

Các phần mềm an toàn hầu như luôn chứa rất nhiều chuỗi có ý nghĩa (Con người có thể đọc được). Mã độc nếu đã được packed hoặc obfuscated thường chứa rất ít chuỗi có ý nghĩa. Do đó, nếu tìm kiếm trong một chương trình mà thấy có rất ít chuỗi có ý nghĩa, rất có thể chương trình đó đã được packed hoặc obfuscated, điều này gợi ý nó có thể là một mã độc và ta có thể sẽ mất nhiều công sức hơn là phân tích tĩnh để tìm hiểu nó.

#### **Đóng gói các files**

Việc đóng gói bắt nguồn từ mục đích giảm kích thước file, làm cho việc tải file nhanh hơn. Nhà phát triển phần mềm thường đóng gói các file nhằm gây khó khăn cho các cracker/reverser/hacker trong việc bẻ khóa hoặc dịch ngược mã nguồn. Đối với những nhà phát triển mã độc, đóng gói làm cho quá trình phân tích tĩnh gặp nhiều khó khăn hơn.

Khi một chương trình đã đóng gói được thực thi, bản chất là một đoạn chương trình mỗi ban đầu sẽ được thực thi (wrapper), chương trình này sẽ thực hiện giải nén (unpacked) chương trình thật, nạp lên bộ nhớ và chuyển điều khiển về cho chương trình chính. Khi một chương trình đã đóng gói được phân tích tĩnh, thực ra chỉ có wrapper là thứ mà ta có thể mở xẻ.

Khi sử dụng, packer nén, mã hóa, lưu hoặc giấu code gốc của chương trình, tự động bổ sung một hoặc nhiều section, sau đó thêm đoạn code Wrapper (Unpacking Stub) và chuyển hướng con trỏ lệnh (Entry Point – EP) tới vùng code này để thực thi. Bình thường, một file không đóng gói (Nonpacked) sẽ được nạp trực tiếp bởi hệ điều hành (OS). Với file đã đóng gói thì Wrapper sẽ được tải bởi OS, sau đó Wrapper sẽ tải chương trình gốc. Lúc này, EP của file thực thi sẽ trỏ tới Wrapper thay vì trỏ vào code gốc.

Để phát hiện các trình đóng gói, chúng ta có thể sử dụng các công cụ: PEiD, CFF Explorer, ExeScan... Một khi chương trình được đóng gói, ta phải giải nén nó (unpack) mới có thể thực hiện bất kỳ thao tác phân tích tĩnh nào.

### **2.3.5. Phân tích PE file Header**

Thường thì các công cụ có thể quét các file thực thi mà không quan tâm định dạng của chúng. Nhưng việc biết được một file thuộc định dạng nào có thể cung cấp nhiều thông tin hữu ích về các chức năng của file thực thi đó.

PE (Portable Executable) là định dạng được sử dụng bởi các file thực thi, đối tượng code hoặc các DLL trong Windows. Định dạng PE là một cấu trúc dữ liệu chứa các thông tin cần thiết để hệ điều hành quản lý các mã thực thi. Gần như tất cả các file có mã thực thi được hệ điều hành Windows tải đều là các file định dạng PE. Tất nhiên, một số định dạng khác vẫn được phát hiện trong một số ít mã độc.

Các file PE bắt đầu bằng một header chứa thông tin về mã nguồn, loại phần mềm, các hàm thư viện cần thiết, không gian cần thiết,... Các thông tin này là rất giá trị đối với việc phân tích mã độc.

Một số công cụ ta có thể sử dụng để phân tích PE header như CFF Explorer và PE Studio.

### **PE header**

PE header chứa rất nhiều thông tin quan trọng trong việc phân tích mã độc, PE header sẽ giúp ta biết được mã độc tương tác với hệ điều hành như thế nào. Trong này còn có một số trường giúp chúng ta xác định được timestamp và location, tức là ta sẽ biết được mã độc được tạo ở đâu và khi nào.

- Containers: Chứa tất cả thông tin mà Windows cần để thực thi chương trình, nó xác định cần phải load DLL nào, vị trí nào là executable code, vị trí nào là data, chương trình này có hỗ trợ GUI hay không? Nếu không có PE header, chương trình sẽ không thể chạy.
- Director: chỉ cho hệ điều hành biết những phần của code được đặt ở đâu trong bộ nhớ RAM.

### **PE sections**

PE sections dùng để xác định từng phần của file PE. Một số loại section như bảng dưới:

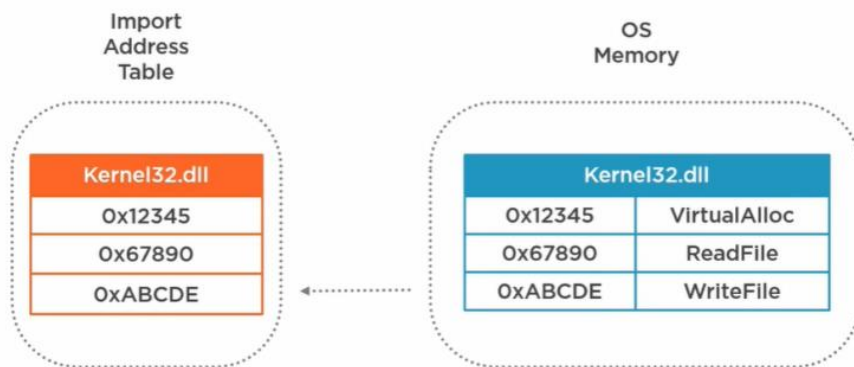
| Name   | Purpose         |
|--------|-----------------|
| .code  | Executable code |
| .text  | Executable code |
| .data  | Program Data    |
| .rdata | Program Data    |
| .idata | Import Table    |
| .edata | Export Table    |
| .rsrc  | Resources       |

### **Import Address Table**

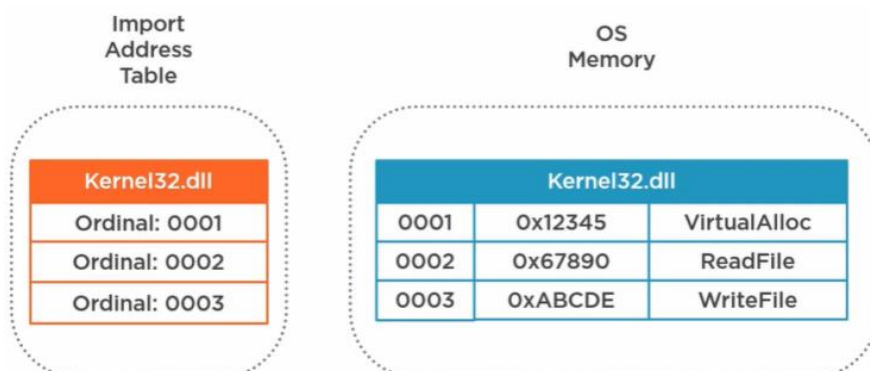
Import Address Table (IAT) chứa danh sách những DLL và API được load bởi chương trình khi thực thi.



Ở hình trên, ở bên trái là IAT, chứa DLL (Kernel32.dll) và API (VirtualAlloc, ReadFile, WriteFile) mà chương trình sử dụng. Khi DLL được load, nó sẽ nằm tại một vị trí nào đó trong bộ nhớ RAM mà chương trình không thể biết trước. Ở hình bên phải, khi Kernel32.dll được load vào trong bộ nhớ, mỗi function sẽ nằm tại một địa chỉ xác định, và loader sẽ thay thế tên của API thành địa chỉ của nó.



Ngoài ra, không nhất thiết phải thay thế tên function bằng địa chỉ của nó mà có thể thay thế bằng Ordinals. Thực chất nó chỉ là một số hiệu định danh function ở trong DLL.



Một số API phổ biến:

- Memory Operations: VirtualAlloc, VirtualProtect, VirtualFree

- File Operations: CreateFile, ReadFile, WriteFile, DeleteFile
- Registry Operations: RegCreateKey, RegDeleteKey, RegGetValue, RegSetValueEx, RegSetKeyValue
- Network Operations: connect, accept, send, recv, listen, InternetConnect, InternetReadFile, InternetWriteFile, gethostbyaddr, gethostbyname
- Misc: LoadLibrary, GetProcAddress, IsDebuggerPresent, WriteProcessMemory, CreateThreadProcess

## **Resource**

Resource là raw data, là bất kì thành phần nào cần thiết để chương trình khởi chạy. Ví dụ như icon, hình ảnh, font chữ, menu, ... Mã độc có thể lưu trữ những đoạn code ẩn hoặc cấu hình ẩn, hoặc những document giả mạo.

### **2.3.6. Dịch ngược mã máy**

Kỹ thuật dịch ngược mã máy (reverse-engineering) được thực hiện bằng cách tải lên các file thực thi, xem xét các chương trình hướng dẫn để khám phá các chương trình bên trong. Dịch ngược mã máy sẽ đưa ra các thông tin chính xác về các chương trình được chạy như thế nào. Tuy nhiên, phương pháp này khó hơn các phương pháp phân tích tĩnh cơ bản rất nhiều, đòi hỏi phải có kiến thức về mảng disassembly, lập trình và các khái niệm về hệ điều hành.

Disassembly cho phép nhà phân tích dịch ngược mã máy của một tập tin thực thi thành mã assembly, trong khi Decompilation dịch ngược mã này thành mã nguồn cao cấp hơn, như C++ hay Java. Công cụ như IDA Pro, Ghidra, Cutter và Radare2 thường được sử dụng trong quá trình này.

## **2.4. Môi trường thực hiện việc phân tích mã độc**

Môi trường ảo ở đây sẽ được xây dựng tùy trường hợp, tùy từng đơn vị phân tích.

Đối với người phân tích mã độc không chuyên thì môi trường ảo đơn giản chỉ là một VMware hay VirtualBox có cài sẵn Windows, Linux, ... cùng một số ứng dụng thông dụng như Java, Flash, ...

Đối với những chuyên gia phân tích chuyên nghiệp trong các phòng thí nghiệm mã độc họ có thể xây dựng những mô hình phân tích phức tạp hơn như xây dựng nhiều máy ảo chạy trên nhiều nền tảng khác nhau, cho các máy đó thông với nhau để thử nghiệm lây lan trong LAN.

Đối với một chuyên gia phân tích mã độc chuyên nghiệp học sẽ xây dựng những sandbox tự động để tiết kiệm thời gian. Đối với một số hãng lớn họ thực hiện phân tích mã độc theo quá trình một cách tự động, các chuyên gia của họ thường dành thời gian để phân tích thủ công một số mã độc đặc biệt hay phân tích lại các biến thể mới.

Để phân tích mã độc người phân tích cần tạo ra những môi trường ảo để phân tích, đảm bảo mã độc không thể lây nhiễm ra ngoài hay làm hại đến máy tính của chính mình.

#### **2.4.1. Môi trường tải mã độc**

- Môi trường tách biệt khỏi mạng làm việc.
- Môi trường không dùng trình duyệt truy cập vào link tải mã độc và trình duyệt dễ bị tấn công. Sử dụng công cụ riêng biệt để tải mã độc.
- Nên sử dụng môi trường khác với môi trường thực thi mã độc (VD: môi trường Linux).
- Sau khi tải về thì đổi tên, tránh trường hợp vô tình thực thi mã độc.

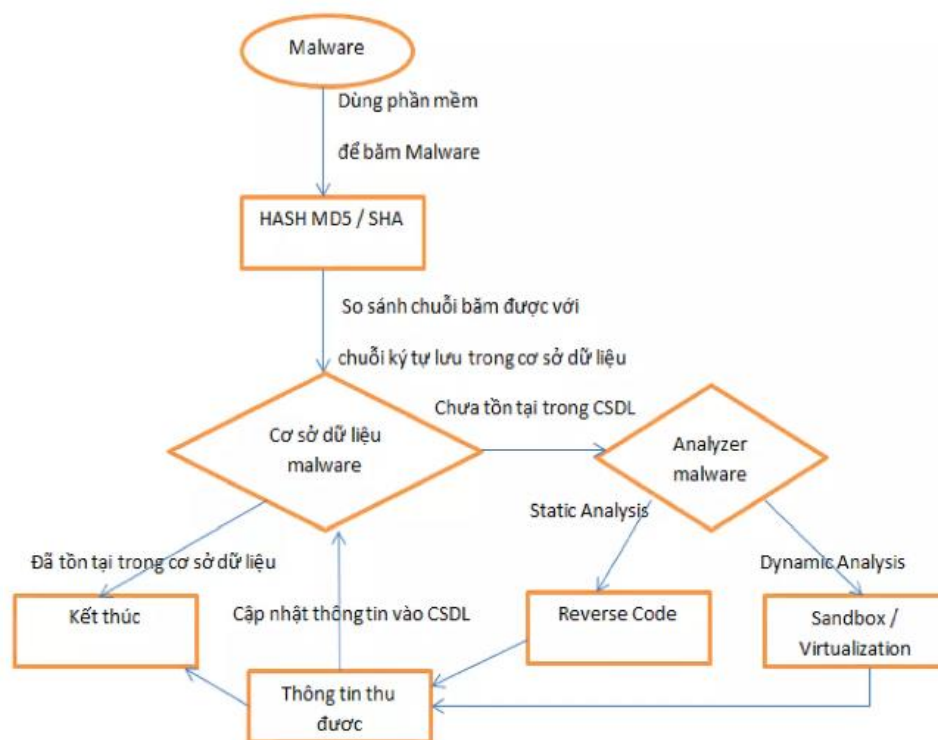
#### **2.4.2. Môi trường phân tích mã độc**

- Để phân tích được Malware, việc đầu tiên là phải có được các mẫu Malware để thực hiện phân tích.
- Hệ điều hành mà chúng ta thử nghiệm Malware: Windows, Linux, MacOS.
- Môi trường để thử nghiệm Malware, có thể sử dụng máy ảo để thử nghiệm. Xây dựng hệ thống lab ảo làm môi trường thử nghiệm Malware. Ngoài ra, có thể sử dụng phần mềm VMware để tạo các PC ảo để thử nghiệm. Hoặc có thể sử dụng sandbox để tạo môi trường thử nghiệm Malware và giám sát các hoạt động của Malware. Thực tế, đôi khi chúng ta cũng cần sử dụng máy thật để thử nghiệm Malware vì thực tế đã xuất hiện một số Malware sẽ rơi vào trạng thái bất hoạt nếu nó phát hiện được nó đang chạy trên môi trường ảo.
- Chuẩn bị các công cụ để thực hiện việc giám sát sự thay đổi các tiến trình trong hệ thống (thường sử dụng trong phân tích động trong phân tích Malware). Sandbox thường được sử dụng để giám sát các hành vi của Malware bằng cách thực thi Malware trong môi trường sandbox.
- Chuẩn bị các công cụ phục vụ cho việc giải mã để phân tích mã nguồn của Malware (thường sử dụng trong phân tích tĩnh trong phân tích Malware).

- Sử dụng tool để lưu giữ các gói tin từ đó phân tích xem liệu Malware có thực hiện kết nối liên lạc ra môi trường Internet hay không.
- Lab phục vụ cho kỹ thuật phân tích gì: tĩnh hay động?
- Sau khi thực hiện phân tích Malware, chúng ta cần phải xây dựng cơ sở dữ liệu về Malware phục vụ cho việc xác định mẫu Malware sau này. Dựa vào dấu hiệu nhận biết Malware hoặc dấu vân tay của Malware.

### 2.4.3. Mô hình môi trường phân tích mã độc

Mô hình tổng quát việc thực hiện phân tích Malware:



Quá trình phân tích phát hiện Malware:

- Malware được đưa qua một phần mềm băm ra lấy mã băm (MD5/SHA).
- Chuỗi băm được so sánh với chuỗi kí tự lưu trong cơ sở dữ liệu.
- Nếu tồn tại trong cơ sở dữ liệu thì đó là Malware và kết thúc quá trình phân tích.
- Nếu không tồn tại trong cơ sở dữ liệu thì sẽ được đẩy sang hệ thống phân tích (Mlalyzer Malware), sử dụng một trong hai phương pháp phân tích tĩnh hoặc phân tích động.

Kết quả của quá trình phân tích đưa ra là phát hiện virus thì mã băm của nó sẽ được lưu vào trong cơ sở dữ liệu và kết thúc quá trình. Nếu không thì kết thúc quá trình phân tích luôn.

#### **2.4.4. Xây dựng hệ thống lab phân tích Malware**

Có thể xây dựng lab trên môi trường thật có nghĩa là sử dụng các thiết bị vật lý hoặc xây dựng trên môi trường ảo sử dụng phần mềm Vmware. Chúng ta sẽ đề cập tới cả hai phương pháp nhưng sẽ tập trung và nhấn mạnh vào việc xây dựng hệ thống phân tích Malware trên môi trường ảo:

- Môi trường vật lý (Physical environment): xây dựng trên môi trường vật lý tức là chúng ta cần có những máy tính thật chạy các hệ điều hành của Windows, Linux, MacOS. Sau đó, chúng ta sẽ thử nghiệm Malware trên chính những chiếc máy tính thật này. Để đảm bảo chiếc PC mà chúng ta thử nghiệm sẽ không bị lỗi, hỏng hóc về phần mềm hoặc hệ điều hành trong quá trình thử nghiệm Malware, nên sử dụng phần mềm đóng băng ổ cứng như Deep Freeze như một cách để backup hệ thống.
- Môi trường ảo (Virtual environment): đây có thể là các máy ảo chạy các hệ điều hành của Windows, Linux, MacOS hoặc sandbox. Chúng ta sẽ thực thi Malware trên những máy ảo này. Trước khi thực thi Malware, chúng ta nên tạo ra một bản snapshot để lưu lại trạng thái của máy ảo trước khi thực thi. Nếu xảy ra lỗi chúng ta chỉ cần quay lại về trạng thái đã đặt snapshot.

Trước khi bắt đầu với việc xây dựng một môi trường lab phải luôn nhớ rằng việc xây dựng một môi trường an toàn là rất quan trọng. Các lỗi hỏng có thể gây lỗi trên máy thật chúng ta xây dựng môi trường lab. Một số lưu ý khi xây dựng môi trường lab an toàn:

- Không chia sẻ tài nguyên giữa máy thật và máy ảo.
- Đảm bảo rằng máy thật được cập nhật những phiên bản mới nhất của các phần mềm anti-virus.
- Nên cài đặt card mạng của máy ảo ở card Nat, sử dụng máy ảo này kết nối Internet và download các Malware cần thiết, không nên sử dụng máy thật để download Malware.
- Ngăn chặn target nhập tới bất cứ thiết bị được chia sẻ (shared devices) hoặc phương tiện di động (removable media) nào, ví dụ USB drives được cắm vào máy thật.



## **PHẦN 3. ỨNG DỤNG PHÂN TÍCH TÌNH TRONG PHÂN TÍCH MALWARE**

### **3.1. Thông tin về mã độc Zeus Banking Trojan**

Zeus Banking Trojan, còn được gọi là Zbot, hay Zeus, là một phần mềm độc hại nổi tiếng được thiết kế để đánh cắp thông tin ngân hàng. Được phát hiện lần đầu vào năm 2007, Zeus nổi tiếng với khả năng ẩn mình và duy trì sự tồn tại, cho phép nó không bị phát hiện trong khi thu thập dữ liệu nhạy cảm như thông tin tài khoản ngân hàng và thẻ tín dụng thông qua các kỹ thuật như ghi lại phím và lấy thông tin từ biểu mẫu.

Zeus thường hoạt động như một phần của một mạng botnet, được điều khiển bởi một máy chủ chỉ huy trung tâm, và có khả năng thực hiện các cuộc tấn công "man-in-the-browser" (MitB), chặn và thao tác các giao dịch trực tuyến trong thời gian thực.

Các phương thức lây nhiễm:

- Email lừa đảo (Phishing Emails): Các tệp đính kèm hoặc liên kết độc hại trong email.
- Tải về tự động (Drive-by Downloads): Tải xuống tự động từ các trang web bị xâm phạm.
- Kỹ thuật xã hội (Social Engineering): Thuyết phục người dùng tải xuống và thực thi phần mềm độc hại.

Tác động: Zeus đã gây ra tổn thất tài chính đáng kể trên toàn cầu, ảnh hưởng đến cả cá nhân và các tổ chức lớn. Mặc dù đã có nhiều nỗ lực từ các cơ quan thực thi pháp luật để triệt phá các mạng botnet Zeus, các biến thể của phần mềm độc hại này vẫn tiếp tục xuất hiện, cho thấy sự kiên cường của nó.

Chiến lược phòng thủ:

- Bảo vệ điểm cuối (Endpoint Protection): Sử dụng các giải pháp diệt virus và phần mềm độc hại mạnh mẽ.
- Bảo mật mạng (Network Security): Giám sát lưu lượng mạng để phát hiện hoạt động độc hại.
- Giáo dục người dùng (User Education): Nâng cao nhận thức về lừa đảo và các thực hành duyệt web an toàn.
- Xác thực hai yếu tố (2FA): Tăng cường bảo mật cho ngân hàng trực tuyến.

- Cập nhật thường xuyên (Regular Updates): Giữ cho hệ thống và phần mềm luôn được cập nhật với các bản vá bảo mật.

Bằng cách hiểu rõ các đặc điểm và phương thức lây nhiễm của Zeus Trojan, chúng ta có thể chuẩn bị tốt hơn để phòng thủ và phân tích mối đe dọa dai dẳng này.

### **3.2. Giới thiệu demo**

Demo hướng dẫn thiết lập một môi trường ảo an toàn để phân tích phần mềm độc hại, cụ thể là mã độc Zeus Banking Trojan. Bên cạnh đó là sử dụng các công cụ và kỹ thuật phân tích tĩnh trong phân tích mã độc để hiểu rõ hơn về mã độc. Mặc dù có nhiều phiên bản của phần mềm độc hại này, chúng ta sẽ tập trung vào một mẫu có khả năng là một trong những biến thể của Zeus Banking Trojan: Gameover Zeus.

Gameover Zeus, còn được gọi là P2PZeus hoặc GOZ, là một loại phần mềm độc hại nổi tiếng xuất hiện lần đầu vào khoảng năm 2011. Đây là một biến thể của Trojan Zeus, chủ yếu nhắm vào các hệ điều hành Windows. Gameover Zeus được thiết kế để đánh cắp thông tin nhạy cảm, chẳng hạn như thông tin tài khoản ngân hàng từ các máy tính bị nhiễm. Nó hoạt động thông qua một mạng peer-to-peer (P2P), giúp nó có khả năng chống lại các nỗ lực triệt phá tốt hơn so với các phiên bản trước đó.

Một trong những tính năng quan trọng nhất của Gameover Zeus là khả năng tạo ra một botnet, một mạng lưới các máy tính bị nhiễm được điều khiển từ xa bởi tội phạm mạng. Botnet này có thể được sử dụng để thực hiện nhiều hoạt động độc hại khác nhau, bao gồm tấn công từ chối dịch vụ phân tán (DDoS), gửi email rác, và lây lan thêm phần mềm độc hại.

### **3.3. Cài đặt VM**

- Cách ly: Đảm bảo phần mềm độc hại được chứa trong môi trường ảo, bảo vệ mạng và hệ thống chủ khỏi nguy cơ phần mềm độc hại thoát ra và lây nhiễm.
- + Đảm bảo cài đặt mạng VM được thiết lập thành: NAT hoặc Host. Chuyển sang Host khi phần mềm độc hại được kích hoạt.
- + Vô hiệu hóa các thư mục chia sẻ giữa máy chủ và VM.
- Ảnh chụp nhanh (Snapshots): Cho phép phục hồi nhanh chóng bằng cách tạo bản sao lưu VM trước các giai đoạn phân tích quan trọng, giúp dễ dàng quay lại các trạng thái trước đó.

### 3.4. Kiến trúc mạng

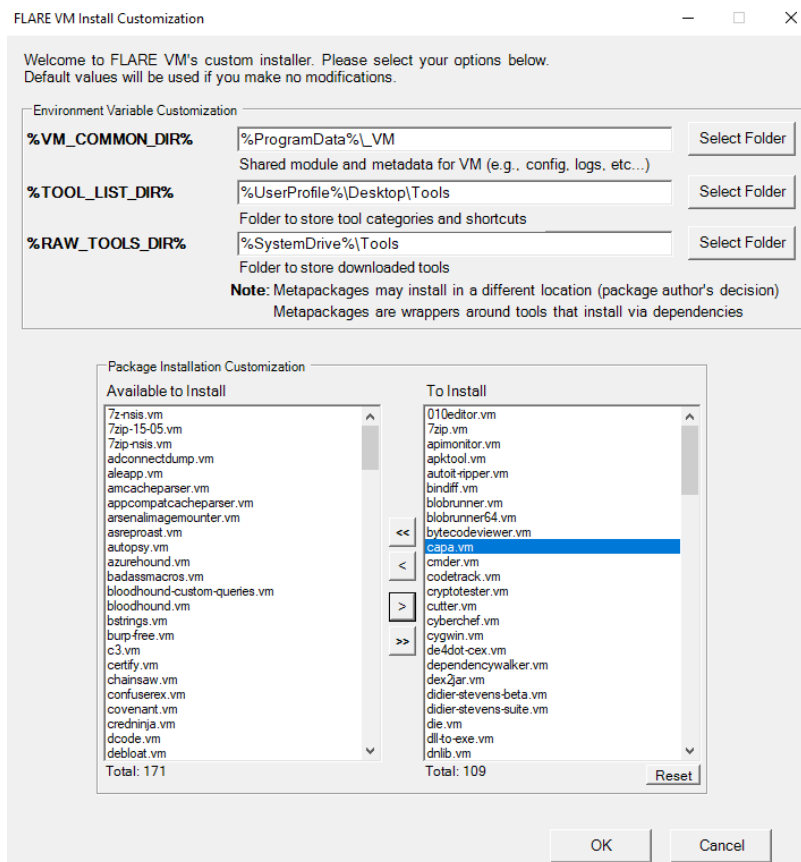
- VMWorkstation: Phần mềm ảo hóa chứa hai máy ảo: Windows 10 và REMnux.
- + Windows 10: Thực thi/phân tích phần mềm độc hại.
- + REMnux: Cấu hình INetSim. (nếu phân tích động)
- Cấu hình Host-only nếu có thể.

### 3.5. Setup

#### 3.5.1. Cài đặt VMWorkstation

Đối với dự án này, VMWorkstation sẽ được sử dụng cho các môi trường ảo của chúng tôi. VMWorkstation cho phép cài đặt dễ dàng các môi trường và tạo ra các mạng để phân tích phần mềm độc hại. Cả Windows 10 và REMnux đều tương thích với phần mềm này.

Hầu hết các công cụ được sử dụng trong phân tích này được sử dụng trong môi trường Windows và có thể dễ dàng có được bằng cách thiết lập Flare VM trong môi trường đó.

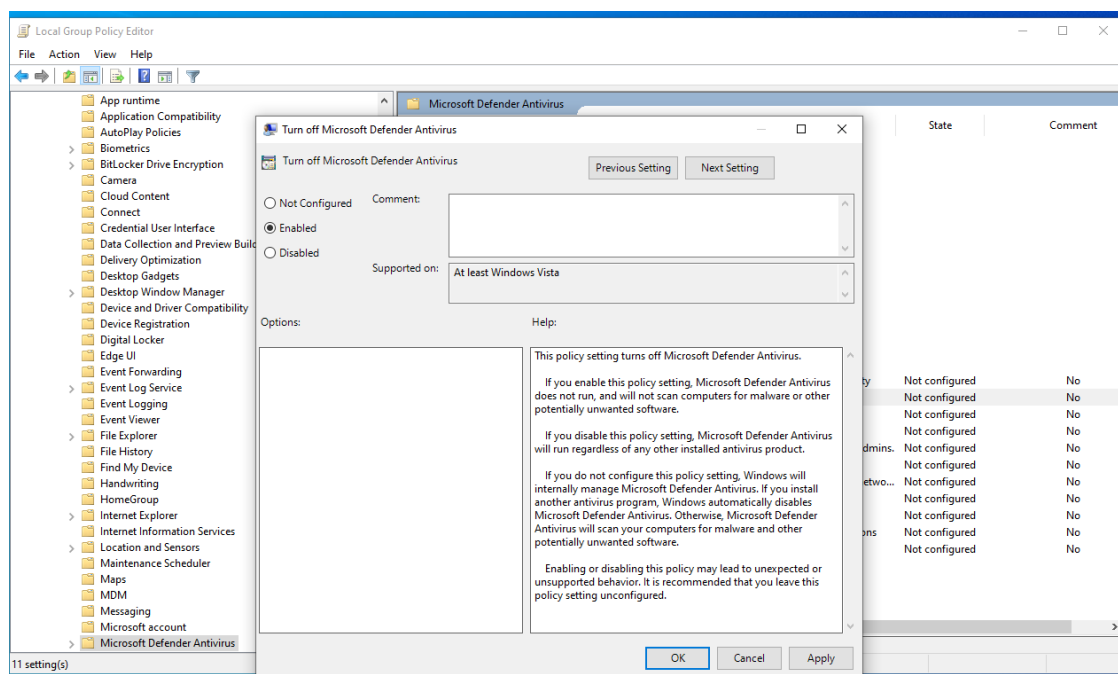


FlareVM là một phân phối bảo mật dựa trên Windows được thiết kế cho phân tích phần mềm độc hại, phản ứng sự cố và kiểm tra xâm nhập. Nó đi kèm với nhiều công cụ và tiện ích đã được cấu hình sẵn, phù hợp cho các nhiệm vụ này, giúp đơn giản hóa quy trình phân tích (đã có sẵn các công cụ PeStudio, Floss và Capa; tự cài đặt thêm Cutter). Các bước cài đặt Cutter:

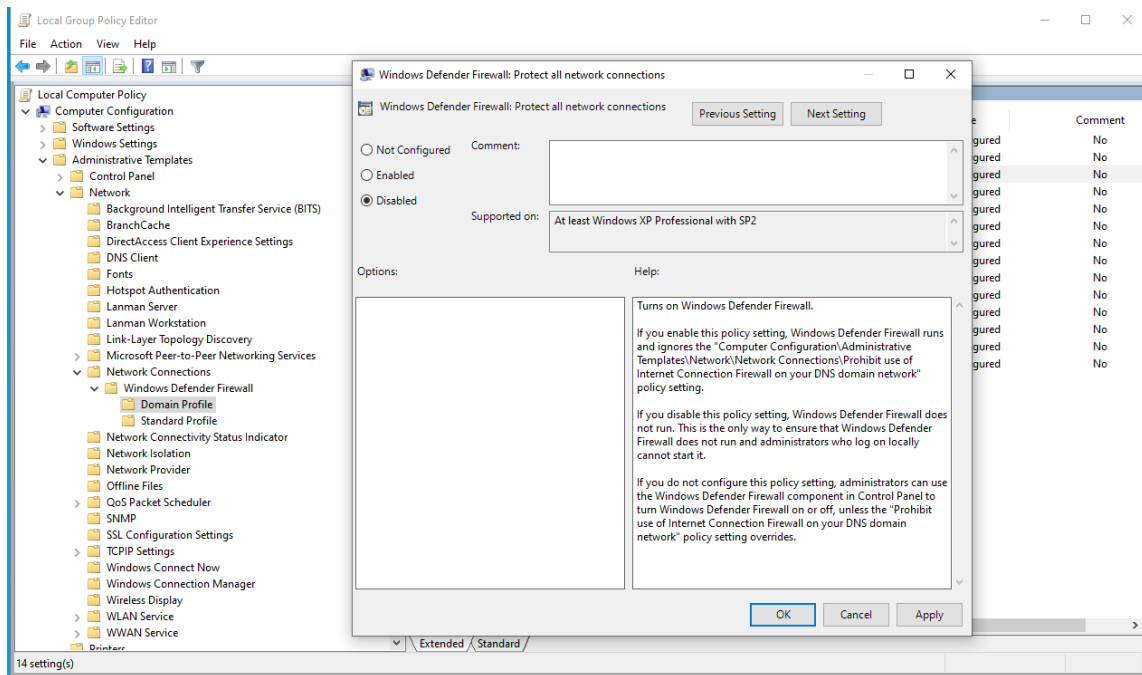
- Truy cập kết nối internet. Tìm kiếm "Cutter download" và tải tệp xuống. Giải nén tệp .zip vào màn hình desktop, sau đó chạy tệp Cutter.exe. Chạy qua tất cả các tùy chọn mặc định và khi nó hỏi "mở tệp" (khi đã có mã độc), chỉ cần chọn tệp invoice\_2318362983713\_823931342io.pdf.exe (tệp chúng ta chọn phân tích) và chờ cho nó chạy tệp .exe.
- Lưu ý các chức năng ở bên trái, và xem entry0. Nếu chúng ta thích xem theo cách trực quan, chọn tùy chọn đồ thị bên dưới.

### 3.5.2. Cài đặt Windows cần thiết cho FlareVM

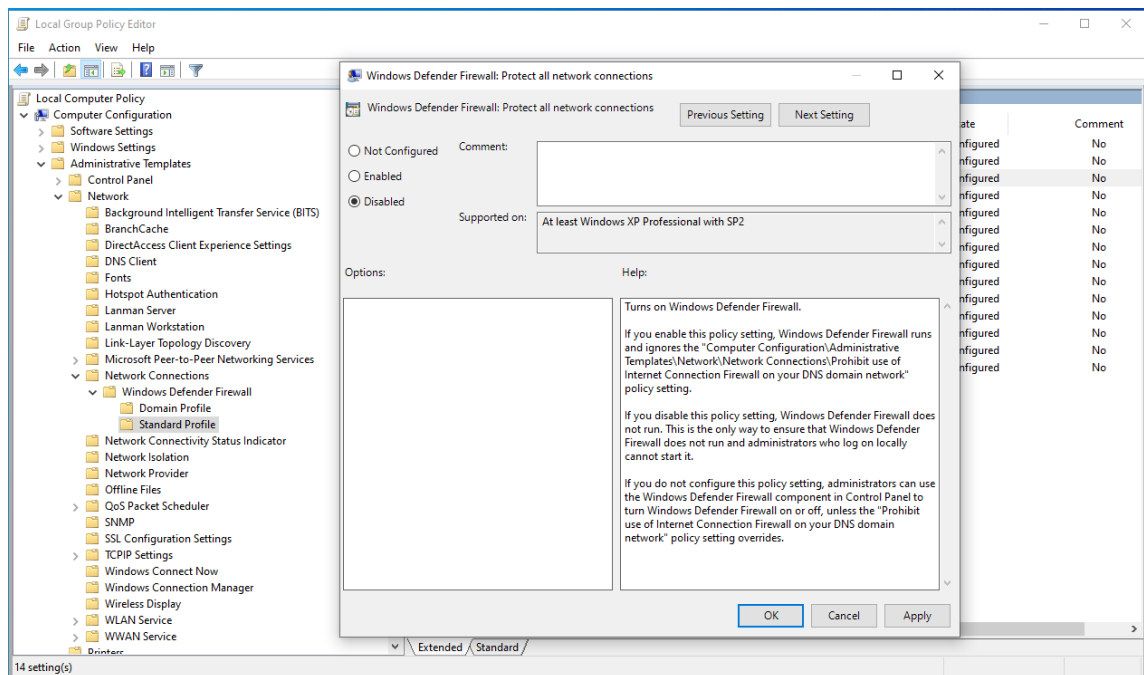
Trước khi thiết lập FlareVM, chúng ta cần vô hiệu hóa tính năng bảo vệ thời gian thực và tắt Windows Defender Antivirus. Để thực hiện điều này, chúng ta vào Local Group Policy Editor > Administrative Templates > Windows Components > Microsoft Defender Antivirus. Và bật tính năng này.



Tiếp theo, Administrative Templates > Network > Network Connections > Windows Defender Firewall > Domain Profile. Tại đây, chúng ta sẽ vô hiệu hóa tùy chọn "Protect all network connections".



Cuối cùng, thực hiện tương tự cho Administrative Templates > Network > Network Connections > Windows Defender Firewall > Standard Profile và vô hiệu hóa tùy chọn này.



### 3.5.3. Cài đặt FlareVM

Khi Windows Security đã được vô hiệu hóa đúng cách, chúng ta có thể tải xuống và cài đặt FlareVM.

Bây giờ để cài đặt FlareVM, chúng ta vào [link sau](#) và làm theo hướng dẫn.

### 3.6. Tải mã độc Zeus Banking Trojan

- Kết nối tạm thời với internet trên FlareVM, sử dụng NAT hoặc Bridged Adapter (FlareVM > Machine > Settings > Network > chọn NAT hoặc Bridge Adapter > OK).
- Trên FlareVM, mở Microsoft Edge và truy cập vào trang web này: [ZeusBankingVersion\\_26Nov2013](#)
  - + Tải xuống tệp ZeusBankingVersion\_26Nov2013.zip và kéo nó ra màn hình desktop.
  - + Ngay sau khi tải xuống, đổi lại chế độ Host-only để cắt đứt kết nối internet.
- Trước khi giải nén tệp .exe từ zip, hãy đảm bảo bạn đang trên máy chủ, không có kết nối internet, và đang tuân thủ tất cả các quy trình an toàn. Khi đã đảm bảo các quy trình an toàn, nhấp chuột phải vào tệp .zip > 7-zip > Open Archive > nhập mật khẩu “infected”.

### 3.7. Các công cụ phân tích tĩnh

- VirusTotal: Dịch vụ trực tuyến miễn phí phân tích các tệp, tên miền, địa chỉ IP và URL nghi ngờ để phát hiện các loại phần mềm độc hại bằng cách tổng hợp dữ liệu từ nhiều sản phẩm diệt virus và công cụ quét trực tuyến.
- PeStudio: Công cụ cho phép các nhà nghiên cứu phần mềm độc hại kiểm tra hoạt động bên trong của các tệp thực thi mà không cần chạy chúng, làm nổi bật các lỗ hổng bảo mật tiềm ẩn, các phần đáng ngờ và chỉ số xâm phạm. Tập hợp siêu dữ liệu (hashes, header, properties, libraries, imports, strings).
- Floss: Tiện ích được thiết kế để tự động giải mã các chuỗi từ mã nhị phân phần mềm độc hại; có thể giúp xác định các máy chủ C2 ẩn, cấu hình phần mềm độc hại và các chỉ số xâm phạm khác.
- Capa: Công cụ phát hiện khả năng trong các tệp thực thi; sử dụng hệ thống dựa trên quy tắc để xác định và phân loại một loạt hành vi trong các chương trình nhị phân. Trong đó, hành vi được ánh xạ theo khung MITRE ATT&CK, với khung MITRE ATT&CK là một hướng dẫn để phân loại và mô tả các cuộc tấn công mạng, về cơ bản là kiến thức và mô hình cơ sở cho hành vi của kẻ thù mạng.
- Cutter: Công cụ được thiết kế để hỗ trợ phân tích tĩnh phần mềm độc hại bằng cách cung cấp khả năng giải mã, đồ họa, kịch bản và gỡ lỗi.

### 3.8. Phân tích mã độc

Để tiến hành phân tích, chúng ta sẽ đi qua ba phần chính là nhận diện, phân tích tĩnh cơ bản và phân tích tĩnh nâng cao.

#### 3.8.1. Nhận diện (Fingerprinting)

Nhận diện là giai đoạn đầu tiên trong phân tích phần mềm độc hại, rất quan trọng để thu thập thông tin nhận dạng như giá trị băm và siêu dữ liệu. Những thuộc tính này phân biệt phần mềm độc hại với các tệp hợp lệ và đặt nền tảng cho các phân tích tiếp theo. Chúng ta sẽ sử dụng công cụ VirusTotal.

##### 3.8.1.1 VirusTotal

68/75 security vendors flagged this file as malicious

Community Score: 68 / 75

Size: 247.00 KB | Last Analysis Date: 4 days ago

invoice\_2318362983713\_823931342io.pdf.exe

peexe detect-debug-environment long-sleeps malware checks-user-input direct-cpu-clock-access self-delete via-tor suspicious-udp persistence

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Popular threat label: trojan.zaccess/sirefef | Threat categories: trojan, dropper | Family labels: zaccess, sirefef, wldcr

Security vendors' analysis

| Vendor             | Detection                         | Vendor      | Detection                       |
|--------------------|-----------------------------------|-------------|---------------------------------|
| AhnLab-V3          | Trojan.Win32.ZAccess.R87034       | Alibaba     | Backdoor.Win32/ZAccess.71cb6d44 |
| AliCloud           | Backdoor.Win/ZAccess.emkb         | ALYac       | Trojan.ZeroAccess.RN            |
| Antiy-AVL          | Trojan[Backdoor]/Win32.ZAccess    | Arcabit     | Trojan.WLDCR.C                  |
| Avast              | Win32:Evo-gen [Trj]               | AVG         | Win32:Evo-gen [Trj]             |
| Avira (no cloud)   | TR/Crypt.XPACK.52658              | BitDefender | Trojan.WLDCR.C                  |
| BitDefenderTheta   | Gen:NN.ZexaF.36812.pyW@aqPTyGbO   | Bkav Pro    | W32.AIDetect/Malware            |
| CrowdStrike Falcon | Win/malicious_confidence_100% (W) | Cybereason  | Malicious.54d20d                |
| Cylance            | Unsafe                            | Cynet       | Malicious (score: 99)           |
| DeepInstinct       | MALICIOUS                         | DrWeb       | BackDoor.Maxplus.14813          |

#### Tên tệp

- Tên tệp: invoice\_2318362983713\_823931342io.pdf.exe
- Có thể thấy mẫu này đang cố gắng mạo danh hóa đơn của một công ty. Phần mở rộng tên tệp được cố ý đặt là .pdf trước .exe để đánh lừa người dùng.
- Tệp này đã được 68 tổ chức đánh dấu là độc hại.

#### Các giá trị băm





- Chữ ký MZ: Chuỗi ASCII “MZ” (mã hex: 4D 5A) xuất hiện ở đầu các tệp này. Chữ cái “MZ” đại diện cho Mark Zbikowski, một trong những nhà phát triển hàng đầu của MS-DOS.
- Định dạng tệp: Chữ ký này liên quan đến định dạng thực thi DOS MZ, được sử dụng cho các tệp .EXE trong DOS. Khi được xem dưới dạng văn bản, nội dung của các tệp như vậy là không thể hiểu được, vì chúng không được thiết kế để đọc theo cách đó.

Điều này cung cấp cho chúng ta một lý do khác để kiểm tra tệp dưới các công cụ kiểm tra Portable Executable như PeStudio. PeStudio đã được cài đặt sẵn cùng với FlareVM.

### 3.8.2. Phân tích tĩnh cơ bản (Basic static analysis)

Vậy với phân tích tĩnh cơ bản, điều chúng ta đang cố gắng làm là kiểm tra chương trình hoặc mã và xác định bất kỳ thành phần độc hại nào mà không cần chạy chương trình thực tế. Chúng ta sẽ sử dụng một vài công cụ khác nhau để thu thập thông tin.

#### 3.8.2.1 PeStudio

Khi chúng ta đang trong PeStudio, chúng ta sẽ cố gắng thu thập thêm các thông tin về phần mềm độc hại. Chúng ta sẽ tìm kiếm một số chuỗi thú vị có thể nổi bật, chẳng hạn như tên miền, địa chỉ IP hoặc có thể là một số loại imports khác như DLLs.

#### Thời gian biên dịch và xuất

| stamps         |                                |
|----------------|--------------------------------|
| compiler-stamp | Mon Nov 25 10:32:03 2013 (UTC) |
| debug-stamp    | n/a                            |
| resource-stamp | n/a                            |
| import-stamp   | n/a                            |
| export-stamp   | Mon Nov 25 10:32:01 2013 (UTC) |

Trong đó:

- compiler-stamp (thời gian biên dịch): ngày 25 tháng 11 năm 2013, 10:32:03 UTC. Điều này cho biết tệp nhị phân đã được biên dịch vào thời điểm này (thời điểm mà mã nguồn được chuyển đổi thành mã nhị phân hoặc tệp thực thi bởi trình biên dịch).
- export-stamp (thời gian xuất): ngày 25 tháng 11 năm 2013, 10:32:01 UTC. Điều này chỉ ra rằng thông tin xuất đã được ghi lại vào thời điểm này, gần như cùng thời điểm với compiler-stamp.

## Tên miền

| names                       |                                                                  |
|-----------------------------|------------------------------------------------------------------|
| file                        | c:\users\admin\desktop\invoice_2318362983713_823931342io.pdf.exe |
| debug                       | n/a                                                              |
| export > original-file-name | corect.com                                                       |
| version                     | n/a                                                              |
| manifest                    | n/a                                                              |
| .NET > module               | n/a                                                              |
| certificate > program-name  | n/a                                                              |

Trên màn hình PeStudio, có export URL tên là corect.com bị viết sai chính tả và được nhúng vào tệp này. Tuy nhiên, ta không tìm được thông tin có ý nghĩa nào từ export này.

## Các chỉ số

|                                           | indicator (24)             | detail                                                                 | level |
|-------------------------------------------|----------------------------|------------------------------------------------------------------------|-------|
| indicators (sections > self-modifying)    | sections > self-modifying  | name: .pdata                                                           | +++++ |
| footprints (type > sha256)                | sections > execute         | count: 2                                                               | +++++ |
| virustotal (status > error)               | imports > flag             | PathRenameExtensionA   WinExec   FindNextFileA   GetEnvironmentVari... | +++++ |
| dos-header (size > 64 bytes)              | string > suspicious        | size: 6707 bytes                                                       | ++    |
| dos-stub (size > 152 bytes)               | string > suspicious        | size: 6709 bytes                                                       | ++    |
| rich-header (tooling > Visual Studio 201  | string > suspicious        | size: 6710 bytes                                                       | ++    |
| file-header (executable > 32-bit)         | string > suspicious        | size: 6701 bytes                                                       | ++    |
| optional-header (subsystem > GUI)         | string > suspicious        | size: 6708 bytes                                                       | ++    |
| directories (count > 4)                   | strings > flag             | count: 18                                                              | ++    |
| sections (characteristics > self-modifyir | file > entropy             | 6.982                                                                  | +     |
| libraries (count > 3)                     | file > signature   tooling | Visual Studio 2008                                                     | +     |
| imports (flag > 77)                       | file > sha256              | 69E966E730557FDE8FD84317CDEF1ECE00A8BB3470C0B58F3231E170168A...        | +     |
| exports (n/a)                             | file > size                | 252928 bytes                                                           | +     |
| thread-local-storage (n/a)                | file > type                | executable, 32-bit, GUI                                                | +     |
| .NET (n/a)                                | virustotal > status        | The server name or address could not be resolved                       | +     |
| resources (count > 11)                    | compiler > stamp           | Mon Nov 25 10:32:03 2013                                               | +     |
| strings (flag > 18)                       | resource > items           | count: 11, size: 22178 bytes, file-ratio: 8.77%                        | +     |
| debug (n/a)                               | manifest > general         | name: n/a, description: n/a, level: aslnvoker                          | +     |
| manifest (level > aslnvoker)              | debug                      | n/a                                                                    | +     |
| version (n/a)                             | entry-point > address      | 0x0000A3B6                                                             | +     |
| certificate (n/a)                         | certificate                | n/a                                                                    | +     |
| overlay (n/a)                             | imports > ordinal          | count: 3                                                               | +     |
|                                           | exports                    | n/a                                                                    | +     |
|                                           | overlay                    | n/a                                                                    | +     |

Mục 'Indicators' cung cấp các chỉ số đáng ngờ dựa trên mức độ chỉ số. Tại đây, chúng ta có thể thấy nó cung cấp tất cả thông tin đáng ngờ có thể về mẫu với mức độ chỉ số. Nó cũng hiển thị các kỹ thuật có thể từ khung MITRE ATT&CK, điều này rất hữu ích cho việc phân tích.

## So sánh kích thước thô và kích thước ảo của tệp

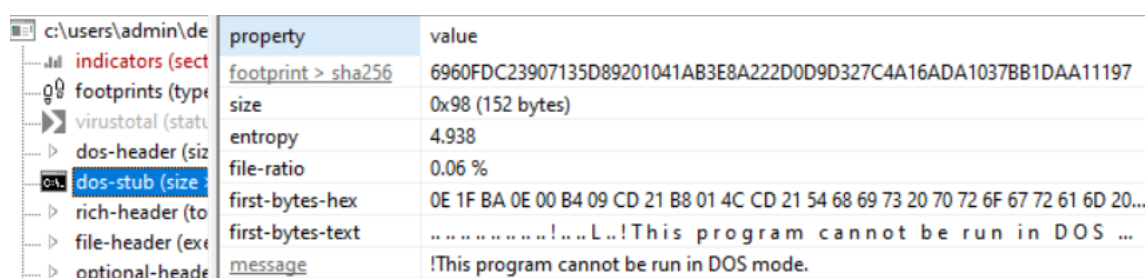
Tiếp theo, chúng ta sẽ vào mục sections từ menu bên trái và so sánh kích thước thô và kích thước ảo của tệp.

| property                    | value                    | value                    | value                   | value                    | value                 | value                   |
|-----------------------------|--------------------------|--------------------------|-------------------------|--------------------------|-----------------------|-------------------------|
| section                     | section[0]               | section[1]               | section[2]              | section[3]               | section[4]            | section[5]              |
| name                        | .text                    | .data                    | .itext                  | .pdata                   | .rsrc                 | .reloc                  |
| footprint > sha256          | 8309B5D320B3D392E2...    | 510A0F9FAF189356CA7...   | 4CDD5D9821CC0790...     | 70CC3E025CCED228E4...    | CB1CB914AD7F61C...    | 7C2F4C4DB94369F90B2A... |
| entropy                     | 6.707                    | 6.130                    | 4.819                   | 6.768                    | 6.143                 | 6.441                   |
| file-ratio (99.60%)         | 18.42 %                  | 30.16 %                  | 1.01 %                  | 38.66 %                  | 9.11 %                | 2.23 %                  |
| raw-address (begin)         | 0x0000400                | 0x0000BA00               | 0x0001E400              | 0x0001EE00               | 0x00036C00            | 0x0003C600              |
| raw-address (end)           | 0x0000BA00               | 0x0001E400               | 0x0001EE00              | 0x00036C00               | 0x0003C600            | 0x0003DC00              |
| raw-size (251904 bytes)     | 0x0000B600 (46592 byt... | 0x00012A00 (76288 byt... | 0x0000A00 (2560 byt...  | 0x00017E00 (97792 byt... | 0x00005A00 (23040 ... | 0x00001600 (5632 bytes) |
| virtual-address             | 0x00001000               | 0x0000D000               | 0x00020000              | 0x00021000               | 0x00039000            | 0x0003F000              |
| virtual-size (250379 byt... | 0x0000B571 (46449 byt... | 0x000128B1 (75953 byt... | 0x0000084D (2125 byt... | 0x00017CBE (97470 byt... | 0x000058F2 (22770 ... | 0x000015EC (5612 bytes) |

So sánh kích thước thô và kích thước ảo của phần mềm độc hại giúp phát hiện việc nén hoặc mã hóa, điều này có thể cho thấy phần mềm độc hại mở rộng trong bộ nhớ, tiết lộ mã hoặc dữ liệu ẩn bổ sung không hiển thị ngay lập tức trong tệp thô. Sự chênh lệch này có thể báo hiệu ý định độc hại và sự hiện diện của các chức năng ẩn. Tại đây, chúng ta thấy vẫn có một chút sự chênh lệch, vì vậy ta chưa khẳng định được tệp có khả năng bị mã hóa hay không.

### Cấu trúc tệp PE (Portable Executable)

1. Dos-header: Xác định tệp là một tệp nhị phân thực thi và cũng chứa các số ma thuật.
2. Dos-stub: Tồn tại để tương thích ngược. Chức năng của nó là in thông điệp.



| property           | value                                                                               |
|--------------------|-------------------------------------------------------------------------------------|
| footprint > sha256 | 6960FDC23907135D89201041AB3E8A222D0D9D327C4A16ADA1037BB1DAA11197                    |
| size               | 0x98 (152 bytes)                                                                    |
| entropy            | 4.938                                                                               |
| file-ratio         | 0.06 %                                                                              |
| first-bytes-hex    | 0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68 69 73 20 70 72 6F 67 72 61 6D 20... |
| first-bytes-text   | .....!...L..!This program cannot be run in DOS ...                                  |
| message            | !This program cannot be run in DOS mode.                                            |

Ở đây chúng ta có thể thấy thông điệp: “!This program cannot be run in DOS mode.”

3. PE header: Xác định tệp thực thi là PE, chứa chữ ký để đại diện cho tệp PE, bao gồm loại máy.
4. Optional header: Kích thước mã (size of the code), địa chỉ điểm vào (address of entry point), địa chỉ cơ sở ưa thích (preferred base address).
5. Sections: Gồm các phân đoạn chứa .text, .bss, .rdata, .data, .rsrc, .edata, .idata, .debug.

### Strings

Tiếp theo, chúng ta sẽ vào phần chuỗi. Đây là một phần rất quan trọng. Tab Strings trong PeStudio giúp xác định các chuỗi có thể đọc được bằng mắt người trong phần mềm độc hại, điều này có thể tiết lộ thông tin quan trọng như URL, địa chỉ IP, đường dẫn tệp, khóa registry, lệnh, và các chỉ số xâm phạm (IOCs) khác. Phân tích các chuỗi này có thể cung cấp cái nhìn sâu sắc về hành vi của phần mềm độc hại, các mục tiêu tiềm năng và kênh giao tiếp, hỗ trợ trong việc hiểu và giảm thiểu mối đe dọa.

| en... | size (...) | location                       | flag (18) | label (75)    | group (12)     | value (1416)              |
|-------|------------|--------------------------------|-----------|---------------|----------------|---------------------------|
| ascii | 7          | <a href="#">section:.itext</a> | x         | -             | execution      | WinExec                   |
| ascii | 9          | <a href="#">section:.itext</a> | x         | -             | input-output   | VkKeyScan                 |
| ascii | 9          | <a href="#">section:.itext</a> | x         | import        | file           | WriteFile                 |
| ascii | 12         | <a href="#">section:.itext</a> | x         | -             | file           | FindNextFile              |
| ascii | 13         | <a href="#">section:.itext</a> | x         | -             | sharing        | GlobalAddAtom             |
| ascii | 14         | <a href="#">section:.itext</a> | x         | import        | memory         | VirtualQueryEx            |
| ascii | 16         | <a href="#">section:.itext</a> | x         | import        | sharing        | GetClipboardData          |
| ascii | 16         | <a href="#">section:.itext</a> | x         | import        | hooking        | GetAsyncKeyState          |
| ascii | 16         | <a href="#">section:.itext</a> | x         | import        | execution      | GetCurrentThread          |
| ascii | 17         | <a href="#">section:.itext</a> | x         | import        | sharing        | GetClipboardOwner         |
| ascii | 18         | <a href="#">section:.itext</a> | x         | import        | sharing        | DdeQueryNextServer        |
| ascii | 19         | <a href="#">section:.itext</a> | x         | -             | file           | PathRenameExtension       |
| ascii | 19         | <a href="#">section:.itext</a> | x         | -             | -              | SetCurrentDirectory       |
| ascii | 20         | <a href="#">section:.itext</a> | x         | import        | sharing        | EnumClipboardFormats      |
| ascii | 22         | <a href="#">section:.itext</a> | x         | -             | reconnaissance | GetEnvironmentVariable    |
| ascii | 22         | <a href="#">section:.itext</a> | x         | -             | reconnaissance | GetEnvironmentVariable    |
| ascii | 24         | <a href="#">section:.itext</a> | x         | import        | windowing      | AllowSetForegroundWindow  |
| ascii | 25         | <a href="#">section:.itext</a> | x         | -             | console        | GetConsoleAliasExesLength |
| ascii | 3          | <a href="#">section:.text</a>  | -         | format-string | -              | %Sz                       |
| ascii | 3          | <a href="#">section:.text</a>  | -         | format-string | -              | :%S                       |
| ascii | 3          | <a href="#">section:.text</a>  | -         | -             | -              | SB+                       |
| ascii | 3          | <a href="#">section:.text</a>  | -         | -             | -              | Eso                       |
| ascii | 3          | <a href="#">section:.text</a>  | -         | -             | -              | 5-h                       |
| ascii | 3          | <a href="#">section:.text</a>  | -         | -             | -              | -z^                       |
| ascii | 3          | <a href="#">section:.text</a>  | -         | -             | -              | ,X:                       |
| ascii | 3          | <a href="#">section:.text</a>  | -         | -             | -              | _^[                       |
| ascii | 3          | <a href="#">section:.text</a>  | -         | -             | -              | -pN                       |
| ascii | 3          | <a href="#">section:.text</a>  | -         | -             | -              | ke-                       |
| ascii | 3          | <a href="#">section:.text</a>  | -         | -             | -              | KUP                       |

Các phát hiện có dấu ‘x’ ở cột flag chỉ ra các chương trình độc hại đã được đánh dấu bởi PeStudio. Đây cơ bản là các API của Windows.

Các cuộc gọi API: Đây là một tập hợp các hàm và cấu trúc dữ liệu mà một chương trình Windows có thể sử dụng để yêu cầu Windows thực hiện một hành động nào đó, chẳng hạn như mở tệp, hiển thị thông điệp, v.v.

Hầu hết mọi thứ mà một chương trình Windows thực hiện đều liên quan đến việc gọi các hàm API khác nhau. Tập hợp tất cả các hàm API mà Windows cung cấp được gọi là "Windows API". Các cuộc gọi API mà tệp đang thực thi trên Windows 10 được thể hiện trong cột value.

| en... | size (...) | location                       | flag (18) | label (75) | group (12)     | value (1416)              |
|-------|------------|--------------------------------|-----------|------------|----------------|---------------------------|
| ascii | 24         | <a href="#">section:.itext</a> | x         | import     | windowing      | AllowSetForegroundWindow  |
| ascii | 18         | <a href="#">section:.itext</a> | x         | import     | sharing        | DdeQueryNextServer        |
| ascii | 20         | <a href="#">section:.itext</a> | x         | import     | sharing        | EnumClipboardFormats      |
| ascii | 12         | <a href="#">section:.itext</a> | x         | -          | file           | FindNextFile              |
| ascii | 16         | <a href="#">section:.itext</a> | x         | import     | hooking        | GetAsyncKeyState          |
| ascii | 16         | <a href="#">section:.itext</a> | x         | import     | sharing        | GetClipboardData          |
| ascii | 17         | <a href="#">section:.itext</a> | x         | import     | sharing        | GetClipboardOwner         |
| ascii | 25         | <a href="#">section:.itext</a> | x         | -          | console        | GetConsoleAliasExesLength |
| ascii | 16         | <a href="#">section:.itext</a> | x         | import     | execution      | GetCurrentThread          |
| ascii | 22         | <a href="#">section:.itext</a> | x         | -          | reconnaissance | GetEnvironmentVariable    |
| ascii | 22         | <a href="#">section:.itext</a> | x         | -          | reconnaissance | GetEnvironmentVariable    |
| ascii | 13         | <a href="#">section:.itext</a> | x         | -          | sharing        | GlobalAddAtom             |
| ascii | 19         | <a href="#">section:.itext</a> | x         | -          | file           | PathRenameExtension       |
| ascii | 19         | <a href="#">section:.itext</a> | x         | -          | -              | SetCurrentDirectory       |
| ascii | 14         | <a href="#">section:.itext</a> | x         | import     | memory         | VirtualQueryEx            |
| ascii | 9          | <a href="#">section:.itext</a> | x         | -          | input-output   | VkKeyScan                 |
| ascii | 7          | <a href="#">section:.itext</a> | x         | -          | execution      | WinExec                   |
| ascii | 9          | <a href="#">section:.itext</a> | x         | import     | file           | WriteFile                 |
| ascii | 4          | <a href="#">section:.pdata</a> | -         | -          | -              | !!1"                      |
| ascii | 4          | <a href="#">section:.pdata</a> | -         | -          | -              | !!71                      |

Trong cột value, chúng ta có thể thấy nhiều cuộc gọi hàm khác nhau có thể làm sáng tỏ hành vi của tệp. PeStudio cũng đánh dấu một số hàm có thể gây vấn đề, chẳng hạn như “GetClipboardOwner”, “WinExec”, “WriteFile”. Mặc dù nhiều hàm này có thể có mục đích hợp pháp, nhưng sự hiện diện của chúng cùng nhau, đặc biệt là trong cùng một tệp thực thi, có thể gây lo ngại về ý định độc hại tiềm tàng của tệp.

Hơn nữa, các giá trị sau đây cũng chỉ ra mã độc hại tiềm năng:

| encoding (2) | size (bytes) | location                       | flag (18) | label (75) | group (12) | value (1416)                                                   |
|--------------|--------------|--------------------------------|-----------|------------|------------|----------------------------------------------------------------|
| ascii        | 57           | <a href="#">section:.pdata</a> | -         | -          | -          | AskmaceaglyBubuPulsKaifTeesMistPeelGhisPrimChaoLyreroeno       |
| ascii        | 15           | <a href="#">section:.pdata</a> | -         | -          | -          | KERNEL32.MulDiv                                                |
| ascii        | 35           | <a href="#">section:.pdata</a> | -         | -          | -          | BagsSpicDollBikeAzonPoopHamsPyasmap                            |
| ascii        | 28           | <a href="#">section:.pdata</a> | -         | -          | -          | KERNEL32.SetCurrentDirectory                                   |
| ascii        | 11           | <a href="#">section:.pdata</a> | -         | -          | -          | BardHolyawe                                                    |
| ascii        | 20           | <a href="#">section:.pdata</a> | -         | -          | -          | SHLWAPI.SHFreeShared                                           |
| ascii        | 47           | <a href="#">section:.pdata</a> | -         | -          | -          | BathEftsDawnvilepughThroCymakohloverMitefuzerat                |
| ascii        | 28           | <a href="#">section:.pdata</a> | -         | -          | -          | SHLWAPI.PathMakeSystemFolder                                   |
| ascii        | 41           | <a href="#">section:.pdata</a> | -         | -          | -          | BemaCadsPodsWavyCedeRadsbriOustPerefenom                       |
| ascii        | 21           | <a href="#">section:.pdata</a> | -         | -          | -          | USER32.SetDlgItemText                                          |
| ascii        | 33           | <a href="#">section:.pdata</a> | -         | -          | -          | BullbonyaweeWaitsnugTierDriblibye                              |
| ascii        | 21           | <a href="#">section:.pdata</a> | -         | -          | -          | KERNEL32.VirtualQuery                                          |
| ascii        | 14           | <a href="#">section:.pdata</a> | -         | -          | -          | CameValeWauler                                                 |
| ascii        | 15           | <a href="#">section:.pdata</a> | -         | -          | -          | USER32.IsIconic                                                |
| ascii        | 35           | <a href="#">section:.pdata</a> | -         | -          | -          | CedeSalsshullImyThroliraValeDonabox                            |
| ascii        | 18           | <a href="#">section:.pdata</a> | -         | -          | -          | USER32.CreateCaret                                             |
| ascii        | 24           | <a href="#">section:.pdata</a> | -         | -          | -          | CellrotoCrudUntohighCols                                       |
| ascii        | 19           | <a href="#">section:.pdata</a> | -         | -          | -          | KERNEL32.CreateFile                                            |
| ascii        | 25           | <a href="#">section:.pdata</a> | -         | -          | -          | DenyLubeDunssawsOresvarut                                      |
| ascii        | 26           | <a href="#">section:.pdata</a> | -         | -          | -          | SHLWAPI.PathRemoveFileSpec                                     |
| ascii        | 40           | <a href="#">section:.pdata</a> | -         | -          | -          | DragRoutflusCrowPeatmownNewsyaksSerfmare                       |
| ascii        | 18           | <a href="#">section:.pdata</a> | -         | -          | -          | USER32.DestroyIcon                                             |
| ascii        | 11           | <a href="#">section:.pdata</a> | -         | -          | -          | Dumpcotsavo                                                    |
| ascii        | 20           | <a href="#">section:.pdata</a> | -         | -          | -          | USER32.SetDlgItemInt                                           |
| ascii        | 62           | <a href="#">section:.pdata</a> | -         | -          | -          | DungBadebankBangGelthoboCocaBozotsksWheyVaryShoghoseNipsCadisi |
| ascii        | 15           | <a href="#">section:.pdata</a> | -         | -          | -          | USER32.EndPaint                                                |
| ascii        | 58           | <a href="#">section:.pdata</a> | -         | -          | -          | ExitRollWoodGumsgamaSloerevsWussletssinkYearZitiryesHypout     |
| ascii        | 19           | <a href="#">section:.pdata</a> | -         | -          | -          | USER32.GetClassInfo                                            |
| ascii        | 15           | <a href="#">section:.pdata</a> | -         | -          | -          | FociTalcileador                                                |

Những gì chúng ta thấy trong hình ảnh là một tập hợp các giá trị ngẫu nhiên theo sau bởi một hàm. Điều này xảy ra lặp đi lặp lại với nhiều giá trị, có thể chỉ ra việc mã



hóa mã. Đây là khi các tác giả phần mềm độc hại sử dụng các kỹ thuật mã hóa để làm cho mã của họ khó phân tích và hiểu hơn.

Các chuỗi ngẫu nhiên xen kẽ với các hàm có thể là một phần của chiến lược mã hóa nhằm ngụy trang chức năng thực sự của mã. Điều này cũng có thể là một chỉ báo về việc sử dụng các kỹ thuật giải quyết hàm động để tránh phân tích tĩnh. Thay vì gọi các hàm theo tên, phần mềm độc hại có thể xây dựng các con trỏ hàm hoặc sử dụng các phương pháp khác để giải quyết địa chỉ hàm tại thời gian chạy. Trong những trường hợp như vậy, các giá trị ngẫu nhiên có thể là dữ liệu được sử dụng trong quá trình giải quyết.

## DLLs

Tab Libraries cung cấp cái nhìn tổng quan về các thư viện bên ngoài (DLL) mà tệp thực thi đã phân tích phụ thuộc vào. Nó liệt kê tên của các thư viện này, cùng với số lượng hàm được nhập từ mỗi thư viện.

| library (3)                  | duplicate (0) | flag (0) | first-thunk-original (INT) | first-thunk (IAT) | type (1) | imports (77) | group (0) | description                                |
|------------------------------|---------------|----------|----------------------------|-------------------|----------|--------------|-----------|--------------------------------------------|
| <a href="#">SHLWAPI.dll</a>  | -             | -        | 0x00020208                 | 0x00020078        | implicit | <u>21</u>    | -         | Shell Light-weight Utility Library         |
| <a href="#">KERNEL32.dll</a> | -             | -        | 0x00020190                 | 0x00020000        | implicit | <u>29</u>    | -         | Windows NT BASE API Client                 |
| <a href="#">USER32.dll</a>   | -             | -        | 0x00020260                 | 0x000200D0        | implicit | <u>27</u>    | -         | Multi-User Windows USER API Client Library |

Một số thư viện chính bao gồm:

- SHLWAPI.dll: Thư viện này chứa các hàm cho việc thao tác chuỗi. Trong ngữ cảnh này, nó có thể được sử dụng để thao tác với đường dẫn, tệp hoặc các mục trong registry như một phần của hoạt động.
- KERNEL32.dll: Đây là một thư viện cốt lõi của Windows, chứa một loạt các hàm cần thiết cho bất kỳ ứng dụng Windows nào. Trong ngữ cảnh phần mềm độc hại, nó có thể được sử dụng cho việc thao tác bộ nhớ, tiêm tiến trình, kiểm soát thực thi và tương tác với hệ điều hành ở mức thấp.
- USER32.dll: Thư viện User32 cung cấp các hàm để tạo và quản lý cửa sổ, nhận đầu vào từ người dùng, và xử lý thông điệp giữa các ứng dụng. Nó có thể bị sử dụng một cách độc hại để thao tác cửa sổ, thu thập đầu vào của người dùng, hoặc tạo các hộp thoại giả.

Tab DDL này cho biết nếu có thư viện bị trùng lặp, bất kỳ cờ nào và địa chỉ của các hàm đã nhập. Loại "Implicit" có nghĩa là các thư viện này sẽ tự động tải khi khởi động. Số lượng import cho thấy số lượng hàm được nhập từ mỗi DLL. Bằng cách phân tích các thư viện và hàm này, chúng ta có thể xác định các hoạt động quan trọng

của phần mềm độc hại, đối chiếu với các mối đe dọa đã biết và ghi chú các hoạt động đáng ngờ. Sau đây là các cuộc gọi API từ từng thư viện riêng lẻ:

## SHLWAPI.dll

| imports (77)                            | callback (0) | group (0) | technique (7) | type (4) | ordinal (1) | library (3)                 |
|-----------------------------------------|--------------|-----------|---------------|----------|-------------|-----------------------------|
| <a href="#">PathCombineW</a>            |              | file      | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">PathQuoteSpacesA</a>        |              | file      | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">PathRenameExtensionA</a>    |              | file      | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">PathRemoveArgsA</a>         |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">StrCmpNIA</a>               |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">PathMatchSpecW</a>          |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">IsCharSpaceA</a>            |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">PathMakeSystemFolderA</a>   |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">PathIsRelativeA</a>         |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">PathIsSameRootA</a>         |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">PathParseIconLocationW</a>  |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">PathIsUNCServerA</a>        |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">14 (GetAcceptLanguages)</a> |              | -         | -             | implicit | x           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">ChrCmplW</a>                |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">PathAddExtensionW</a>       |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">157 (StrCmpIC)</a>          |              | -         | -             | implicit | x           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">PathIsRootW</a>             |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">29 (IsCharSpace)</a>        |              | -         | -             | implicit | x           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">PathIsPrefixA</a>           |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">PathRelativePathToW</a>     |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |
| <a href="#">ChrCmplA</a>                |              | -         | -             | implicit | -           | <a href="#">SHLWAPI.dll</a> |

## KERNEL32.dll

| imports (77)                               | callback (0) | group (0)      | technique (7)    | type (4) | ordinal (1) | library (3)                  |
|--------------------------------------------|--------------|----------------|------------------|----------|-------------|------------------------------|
| <a href="#">DeleteCriticalSection</a>      |              | synchro        | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GlobalAddAtomA</a>             |              | sharing        | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">SizeofResource</a>             |              | resource       | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetEnvironmentVariableA</a>    |              | reconnaissa... | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetDriveTypeA</a>              |              | reconnaissa... | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetTickCount</a>               |              | reconnaissa... | T1124   Syst...  | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetEnvironmentVariableW</a>    |              | reconnaissa... | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetLogicalDrives</a>           |              | reconnaissa... | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">LocalFree</a>                  |              | memory         | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">LocalAlloc</a>                 |              | memory         | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">VirtualQueryEx</a>             |              | memory         | T1055   Proc...  | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">LocalUnlock</a>                |              | memory         | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">HeapFree</a>                   |              | memory         | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">FindNextFileA</a>              |              | file           | T1083   File ... | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">CreateFileMappingA</a>         |              | file           | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetCompressedFileSizeA</a>     |              | file           | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">WriteFile</a>                  |              | file           | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetPrivateProfileIntW</a>      |              | execution      | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">WinExec</a>                    |              | execution      | T1106   Exec...  | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetCurrentThread</a>           |              | execution      | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">FreeLibrary</a>                |              | dynamic-lib... | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetModuleHandleW</a>           |              | dynamic-lib... | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetConsoleAliasExesLengthW</a> |              | console        | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetUserDefaultUILanguage</a>   |              | -              | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetOEMCP</a>                   |              | -              | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">SetCurrentDirectoryW</a>       |              | -              | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">IsBadReadPtr</a>               |              | -              | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetSystemDefaultUILanguage</a> |              | -              | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |
| <a href="#">GetSystemDefaultLCID</a>       |              | -              | -                | implicit | -           | <a href="#">KERNEL32.dll</a> |

Hình trên cho thấy một số cuộc gọi API quan trọng được phân tích, chẳng hạn như GetTickCount, liên quan đến thời gian hoạt động của máy chủ để phát hiện ra máy ảo.

## USER32.dll

| imports (77)                             | callback (0) | group (0)    | technique (7)   | type (4) | ordinal (1) | library (3)                |
|------------------------------------------|--------------|--------------|-----------------|----------|-------------|----------------------------|
| <a href="#">CallWindowProcW</a>          |              | windowing    | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">UpdateWindow</a>             |              | windowing    | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">IsWindowEnabled</a>          |              | windowing    | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">GetWindowTextLengthW</a>     |              | windowing    | T1010   Win...  | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">AllowSetForegroundWindow</a> |              | windowing    | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">GetClipboardOwner</a>        |              | sharing      | T1115   Clip... | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">GetClipboardData</a>         |              | sharing      | T1115   Clip... | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">EnumClipboardFormats</a>     |              | sharing      | T1115   Clip... | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">DdeQueryNextServer</a>       |              | sharing      | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">SwapMouseButton</a>          |              | input-output | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">VkKeyScanA</a>               |              | input-output | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">GetAsyncKeyState</a>         |              | hooking      | T1056   Inpu... | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">GetCapture</a>               |              | hooking      | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">CopyAcceleratorTableA</a>    |              | hooking      | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">GetProcessDefaultLayout</a>  |              | -            | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">AppendMenuA</a>              |              | -            | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">GetCaretPos</a>              |              | -            | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">GetSysColor</a>              |              | -            | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">DestroyCursor</a>            |              | -            | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">GetScrollInfo</a>            |              | -            | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">FlashWindowEx</a>            |              | -            | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">SetLastErrorEx</a>           |              | -            | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">InflateRect</a>              |              | -            | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">ShowCaret</a>                |              | -            | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">LoadBitmapA</a>              |              | -            | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">DeleteMenu</a>               |              | -            | -               | implicit | -           | <a href="#">USER32.dll</a> |
| <a href="#">HideCaret</a>                |              | -            | -               | implicit | -           | <a href="#">USER32.dll</a> |

Thư viện USER32.dll gọi một số API đáng ngờ như GetCapture và GetClipboardData, cho phép truy cập dữ liệu clipboard và khả năng chụp màn hình.

Các chuỗi được trích xuất từ mẫu phần mềm độc hại cung cấp cái nhìn sâu sắc về hành vi và khả năng tiềm ẩn của nó. Dưới đây là một phân tích ngắn gọn về những chuỗi này:

- Các hàm trong SHLWAPI.dll:
  - + PathRelativePathToW, PathParseIconLocationW, PathCombineW, PathAddExtensionW: Các hàm liên quan đến việc thao tác đường dẫn tệp, cho thấy phần mềm độc hại có thể thay đổi hoặc tạo dựng các đường dẫn tệp.
  - + ChrCmpIA, ChrCmpIW: Các hàm so sánh chuỗi, chỉ ra rằng phần mềm có thể đang so sánh tên tệp hoặc các chuỗi khác.



- + PathIsPrefixA, PathIsRootW, PathIsUNCServerA, PathIsSameRootA, PathIsRelativeA: Các hàm để xác định bản chất của đường dẫn tệp, cho thấy phần mềm đang thực hiện kiểm tra vị trí tệp.
- + PathRenameExtensionA, PathRemoveArgsA, PathQuoteSpacesA, PathMakeSystemFolderA: Các hàm để sửa đổi thuộc tính và đường dẫn tệp.
- + PathMatchSpecW, StrCmpNIA: Các hàm để thực hiện khớp mẫu và so sánh chuỗi.
- Các hàm trong KERNEL32.dll:
  - + LocalUnlock, FreeLibrary, LocalAlloc, LocalFree: Các hàm quản lý bộ nhớ, cho thấy việc cấp phát và giải phóng bộ nhớ động.
  - + GetEnvironmentVariableW, GetEnvironmentVariableA: Các hàm để đọc các biến môi trường, có thể nhằm thu thập thông tin hệ thống.
  - + GetSystemDefaultUILanguage, GetSystemDefaultLCID, GetUserDefaultUILanguage: Các hàm liên quan đến cài đặt ngôn ngữ và vùng miền của hệ thống, có thể được sử dụng cho hành vi cụ thể theo môi trường.
  - + HeapFree, GlobalAddAtomA: Các hàm quản lý bộ nhớ và chuỗi.
  - + GetLogicalDrives, GetDriveTypeA, GetCompressedFileSizeA: Các hàm để thu thập thông tin về các ổ đĩa của hệ thống.
  - + VirtualQueryEx, IsBadReadPtr: Các hàm để kiểm tra và xác thực bộ nhớ.
  - + WriteFile, CreateFileMappingA: Các hàm để ghi tệp và tệp ánh xạ bộ nhớ, cho thấy khả năng thao tác tệp hoặc lưu trữ dữ liệu.
  - + WinExec: Hàm để thực thi một chương trình, gợi ý rằng phần mềm độc hại có thể khởi động các tiến trình khác.
  - + DeleteCriticalSection: Chỉ ra việc sử dụng các cơ chế đồng bộ hóa, có thể nhằm quản lý đồng thời trong đa luồng.
- Các hàm trong USER32.dll:
  - + VkKeyScanA, GetAsyncKeyState: Các hàm liên quan đến đầu vào từ bàn phím, có thể cho thấy khả năng keylogging.

- + GetClipboardOwner, GetClipboardData, EnumClipboardFormats: Các hàm để truy cập dữ liệu từ clipboard, cho thấy malware có thể đang ăn cắp nội dung clipboard.
- + CallWindowProcW, SetLastErrorEx: Các hàm để quản lý thông điệp cửa sổ và xử lý lỗi.
- + AllowSetForegroundWindow, UpdateWindow, FlashWindowEx: Các hàm liên quan đến trạng thái và cập nhật cửa sổ, chỉ ra khả năng thao tác với trạng thái cửa sổ.
- + ShowCaret, HideCaret: Các hàm để thao tác với con trỏ, có thể liên quan đến keylogging hoặc xử lý đầu vào.
- + GetCapture, IsWindowEnabled, IsWindowVisible: Các hàm để tương tác với các thuộc tính cửa sổ, có thể nhằm theo dõi trạng thái cửa sổ.

Sự hiện diện của các hàm này cho thấy phần mềm độc hại có khả năng:

- Thao tác tệp và đường dẫn.
- Quản lý bộ nhớ và tiến trình.
- Thu thập thông tin hệ thống và môi trường.
- Tương tác với và thao tác cửa sổ cũng như đầu vào của người dùng, có khả năng cho hoạt động gián điệp hoặc keylogging.
- Thực thi các tiến trình khác và quản lý tệp.

Điều này chỉ ra rằng phần mềm độc hại rất linh hoạt và có khả năng thực hiện nhiều hoạt động thường liên quan đến hành vi độc hại, chẳng hạn như gián điệp, ăn cắp thông tin và thao tác tài nguyên hệ thống.

### 3.8.2.2 Floss

Đối với công cụ tiếp theo, chúng ta sẽ sử dụng FLOSS, đây là một tiện ích dòng lệnh. Chúng ta sử dụng trình giả lập terminal nâng cao hơn là Cmder. Như đã đề cập trước đó, FLOSS (FireEye Labs Obfuscated String Solver) trích xuất các chuỗi bị mã hóa từ phần mềm độc hại, điều này có thể tiết lộ các lệnh ẩn, URL và dữ liệu hữu ích khác. Vì tệp nhị phân này có khả năng không bị nén, nên chúng ta sẽ không cần sử dụng bất kỳ công cụ hay chiến thuật nào khác.

```
Cmder
C:\Users\admin
λ cd Desktop

C:\Users\admin\Desktop
λ ls
available_packages.txt  config.xml  Cutter-v2.3.4-Windows-x86_64  desktop.ini  fakenet_logs.lnk  install.ps1
invoice_2318362983713_823931342io.pdf.exe  Tools  ZeusBankingVersion_26Nov2013.zip

C:\Users\admin\Desktop
λ floss invoice_2318362983713_823931342io.pdf.exe > strings.txt
```

Và như chúng ta thấy, FLOSS đã trích xuất tất cả các chuỗi vào tệp văn bản mà chúng ta mong muốn. Những chuỗi này giống với những gì chúng ta thấy trong PeStudio.

```
FLARE FLOSS RESULTS (version v3.1.0-0-gdb9af41)

+-----+
| file path | invoice_2318362983713_823931342io.pdf.exe |
| identified language | unknown |
| extracted strings |
|   static strings | 791 (146444 characters) |
|   language strings | 0 ( 0 characters) |
|   stack strings | 2 |
|   tight strings | 0 |
|   decoded strings | 66 |
+-----+

FLOSS STATIC STRINGS (791)

+-----+
| FLOSS STATIC STRINGS: ASCII (790) |
+-----+

!This program cannot be run in DOS mode.
Rich
.text
.data
.itext
.pdata
.rsrc
@.reloc
jrjy
SVW;
OAT+
```

Một điều hữu ích khác về FLOSS là ta có thể trích xuất các chuỗi dựa trên độ dài của chúng dựa trên câu lệnh: ``floss -n 6 invoice_2318362983713_823931342io.pdf.exe``. Câu lệnh trên sẽ trích xuất các chuỗi có 6 ký tự trở lên.

### 3.8.2.3 Capa

Công cụ tiếp theo mà chúng ta sẽ sử dụng là Capa. Chúng ta mở PowerShell và gõ ``capa filename``, filename chúng ta đang phân tích ở đây là “invoice\_2318362983713\_823931342io.pdf.exe”, dưới đây là kết quả:

|                                                                                      |                                                                  |
|--------------------------------------------------------------------------------------|------------------------------------------------------------------|
| PS C:\Windows\system32> cd C:\Users\admin\Desktop                                    |                                                                  |
| PS C:\Users\admin\Desktop> Capa invoice_2318362983713_823931342io.pdf.exe            |                                                                  |
| md5                                                                                  | ea039a854d20d7734c5add48f1a51c34                                 |
| sha1                                                                                 | 9615dca4c0e46b8a39de5428af7db060399230b2                         |
| sha256                                                                               | 69e966e730557fde8fd84317cdef1ece00a8bb3470c0b58f3231e170168af169 |
| analysis                                                                             | static                                                           |
| os                                                                                   | windows                                                          |
| format                                                                               | pe                                                               |
| arch                                                                                 | i386                                                             |
| path                                                                                 | C:/Users/admin/Desktop/invoice_2318362983713_823931342io.pdf.exe |
|                                                                                      |                                                                  |
| ATT&CK Tactic                                                                        | ATT&CK Technique                                                 |
| DEFENSE EVASION                                                                      | Virtualization/Sandbox Evasion::System Checks T1497.001          |
|                                                                                      |                                                                  |
| MBC Objective                                                                        | MBC Behavior                                                     |
| ANTI-BEHAVIORAL ANALYSIS                                                             | Virtual Machine Detection [B0009]                                |
|                                                                                      |                                                                  |
| Capability                                                                           | Namespace                                                        |
| reference anti-VM strings targeting VMWare<br>resolve function by parsing PE exports | anti-analysis/anti-vm/vm-detection<br>load-code/pe               |

Dựa trên mẫu phần mềm độc hại của chúng ta, Capa đã liên kết nó với kỹ thuật Defense Evasion (Tránh phát hiện), trong đó nó cố gắng phát hiện và tránh các môi trường ảo hóa và phân tích. Thông thường, các tác giả phần mềm độc hại sẽ cố gắng lập trình phần mềm của họ theo cách mà nó sẽ cố gắng phát hiện và tránh ảo hóa. Nếu phần mềm phát hiện môi trường máy ảo, nó có thể ngừng hoạt động và giữ im lặng, dẫn đến việc phân tích phần mềm độc hại kém hiệu quả hơn.

Trường MBC Objective đề cập đến MBC Behavior (Danh mục Hành vi Phần mềm độc hại), là một bộ sưu tập các hành vi đã được tài liệu hóa mà các loại phần mềm độc hại khác nhau thể hiện. Những danh mục này phục vụ như các tài liệu tham khảo cho các nhà phân tích để xác định và phân loại phần mềm độc hại dựa trên các hành động và đặc điểm quan sát được của chúng, chẳng hạn như Virtual Machine Detection (Phát hiện máy ảo). Sự hiện diện của khả năng phát hiện máy ảo trong một tệp cho thấy rằng tệp này đang sử dụng các kỹ thuật tránh phát hiện để tránh bị phát hiện và phân tích trong các môi trường sandbox.

Dựa trên kết quả của phần mềm độc hại này, các khả năng chi tiết là:

- Reference anti-VM strings targeting VMWare (Chuỗi Chống VM Nhắm Đến VMWare): Chỉ ra rằng phần mềm độc hại chứa các chuỗi được sử dụng đặc biệt để phát hiện sự hiện diện của VMWare, đây là một kỹ thuật phổ biến để tránh phân tích trong các môi trường ảo hóa.

- Resolve function by parsing PE exports (Giải Quyết Hàm Bằng Cách Phân Tích Xuất PE): Chứng minh rằng phần mềm độc hại có thể giải quyết động các hàm bằng cách phân tích bảng xuất của Portable Executable, một phương pháp thường được sử dụng để mã hóa ý định của nó và vượt qua một số cơ chế bảo mật nhất định.

Chúng ta cũng có thể sử dụng các chế độ chi tiết của Capa với tùy chọn ``-v`` và ``-vv``. Tùy chọn ``-vv`` trong lệnh Capa tăng mức độ chi tiết của đầu ra, cung cấp thông tin chi tiết hơn về quy trình phân tích. Điều này bao gồm ngữ cảnh bổ sung về các quy tắc đang được khớp, các tính năng đang được trích xuất, và các chi tiết cụ thể hơn về khả năng của phần mềm độc hại. Điều này rất hữu ích cho việc kiểm tra sâu hơn và hiểu cách Capa đưa ra kết luận của mình.

Nếu chúng ta thực hiện lệnh Capa với đầu ra rất chi tiết bằng cách sử dụng tùy chọn ``-vv``, chúng ta sẽ nhận được nhiều đầu ra khác nhau như:

```
resolve function by parsing PE exports
namespace load-code/pe
author sara-rn
scope function
function @ 0x40A3B6
and:
  os: windows
or:
  characteristic: loop @ 0x40A3B6
  mnemonic: movzx @ 0x40A7E6, 0x40A883, 0x40AAFD, 0x40AB09, and 19 more...
and:
  offset: 0x30 = IMAGE_DOS_HEADER.PE.e_lfanew @ 0x40A5AF, 0x40A7A7, 0x40AAB5, 0x40ACA7, and 2 more...
or:
  and:
    arch: i386
    offset: 0x70 = offset to IMAGE_DATA_DIRECTORY[IMAGE_DIRECTORY_ENTRY_EXPORT] @ 0x40AC70, 0x40B067, 0x40B097
  3 or more:
    offset: 0x14 = IMAGE_EXPORT_DIRECTORY.NumberOfFunctions @ 0x40A44E, 0x40A4A2, 0x40A539, 0x40A5DF, and 21 more...
    offset: 0x20 = IMAGE_EXPORT_DIRECTORY.AddressOfNameOrdinals @ 0x40A635, 0x40A8FC, 0x40AFDA, 0x40B766, and 1 more...

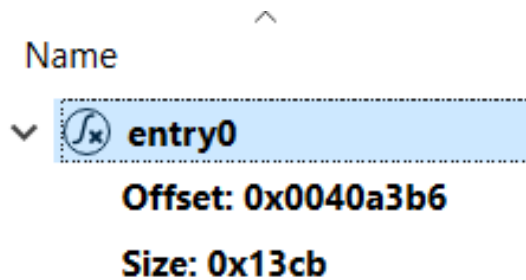
    offset: 0x20 = IMAGE_EXPORT_DIRECTORY.AddressOfNames @ 0x40A676, 0x40A71E, 0x40A9D8, 0x40AAC3, and 12 more...
    offset: 0x10 = IMAGE_EXPORT_DIRECTORY.NumberOfNames @ 0x40A4B8, 0x40A5A3, 0x40A623, 0x40A7C4, and 21 more...
    offset: 0x10 = IMAGE_EXPORT_DIRECTORY.AddressOfFunctions @ 0x40A558, 0x40A560, 0x40A564, 0x40A5B7, and 19 more...
```

Tại đây, chúng ta có thể thấy chính xác trong mã nhị phân, Capa tin rằng tệp đang thể hiện hành vi đã được nêu. Phân tích thêm về địa chỉ bộ nhớ được làm nổi bật 0x40A3B6 sẽ được tiếp tục trong phần tiếp theo của chúng ta.

### 3.8.3. Phân tích tĩnh nâng cao (Advanced static analysis)

#### 3.8.3.1 Cutter

Để có cái nhìn sâu sắc hơn về những gì tệp này có thể thực hiện, chúng ta sẽ sử dụng công cụ Cutter để xem mã đã được giải mã của tệp. Khi xem mã đã được giải mã, dòng được gán nhãn là "entry" chỉ ra điểm bắt đầu thực thi cho chương trình. Đây là một điểm tham chiếu quan trọng cho việc phân tích và hiểu luồng chương trình. Nhãn entry này thường liên quan đến một địa chỉ bộ nhớ nơi các lệnh assembly tương ứng nằm trong đoạn mã của chương trình.



Thật trùng hợp, địa chỉ bộ nhớ bị đánh dấu 0x40A3B6 chính là entry, và do đó, là điểm khởi đầu trong quá trình thực thi của chương trình này.

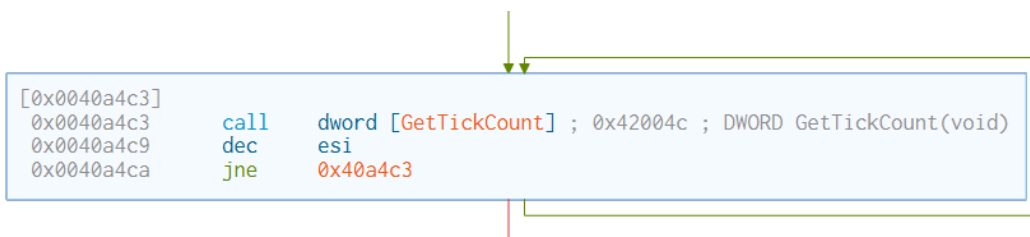
Từ đây, chúng ta có thể sử dụng chức năng đồ thị trong Cutter để có cái nhìn tổng quan hơn về hàm entry. Mỗi hộp chữ nhật trong đồ thị đại diện cho một khối lệnh assembly cơ bản; một chuỗi các lệnh bắt đầu từ một điểm cụ thể và chạy tuần tự đến lệnh cuối cùng, nơi nó có thể nhảy đến một khối khác dựa trên điều kiện hoặc tiếp tục đến khối tiếp theo một cách tuần tự. Mã assembly khi được xem trong Cutter thường có định dạng như sau:

|         | Address    | Instruction | Operands | Comments   |
|---------|------------|-------------|----------|------------|
| Example | 0x0040A3B6 | push        | ebp      | ; 0x40fc8c |

Trong đó:

- Address (Địa chỉ): Vị trí trong bộ nhớ nơi lệnh cư trú, thường được biểu diễn dưới định dạng thập lục phân; cung cấp một định danh duy nhất cho vị trí của từng lệnh trong tệp.
- Instruction (Lệnh): Lệnh ngôn ngữ assembly cho biết CPU cần thực hiện thao tác gì. Có thể thay đổi rất nhiều, từ việc di chuyển dữ liệu giữa các thanh ghi hoặc bộ nhớ đến việc điều khiển luồng của chương trình.
- Operands (Toán hạng): Các mục dữ liệu được lệnh tác động đến. Có thể là các giá trị, thanh ghi CPU và/hoặc địa chỉ bộ nhớ. Có thể có nhiều toán hạng.
- Comments (Chú thích): Dấu chấm phẩy bắt đầu một chú thích trong ngôn ngữ assembly. Những chú thích này không phải là một phần của mã máy nhưng được thêm vào để cung cấp giải thích.

Trong hàm entry, chúng ta tìm thấy lệnh thú vị sau đây:



Địa chỉ 0x0040a4c3 có thể là lý do tại sao Capa đánh dấu hàm tương ứng là đáng ngờ trong việc Defense Evasion (Tránh Phát hiện). Trong khối thứ năm của các lệnh assembly, chúng ta tìm thấy toán hạng [GetTickCount], có thể được sử dụng để lấy số mili giây đã trôi qua kể từ khi hệ thống được khởi động. Mặc dù hàm API này không vốn dĩ độc hại, nhưng nó đã được chứng minh là được sử dụng cho các kỹ thuật tránh phát hiện dựa trên thời gian. Các sandbox thường có giới hạn về thời gian mà chương trình có thể chạy hoặc số lượng thao tác mà nó có thể thực hiện trong một khoảng thời gian nhất định. Bằng cách sử dụng GetTickCount để đo khoảng thời gian, phần mềm độc hại có thể trì hoãn việc thực thi các hành vi độc hại cho đến khi môi trường sandbox đạt được các điều kiện cụ thể của nạn nhân.

Trong trường hợp cụ thể này, chúng ta thấy một lệnh gọi đến GetTickCount, lấy một giá trị tính bằng mili giây và lưu trữ nó trong một thanh ghi để sử dụng sau này. Sau đó, một lệnh giảm giá trị một được thực hiện trên một giá trị nằm trong thanh ghi ESI (một thanh ghi thường được sử dụng làm bộ đếm vòng lặp). Tiếp theo, có một kiểm tra để xem liệu giá trị ESI có không bằng số không hay không. Nếu giá trị bằng không, thì nó nhảy đến khối lệnh mã tiếp theo. Nhưng nếu giá trị không bằng không, thì một vòng lặp sẽ được bắt đầu cho đến khi điều kiện được thỏa mãn.

Bằng cách kiểm tra liên tục thời gian hoạt động của hệ thống hoặc thời gian đã trôi qua bằng cách sử dụng GetTickCount, phần mềm độc hại có thể trì hoãn các hành vi độc hại của nó cho đến khi một điều kiện nhất định được thỏa mãn.

Việc tiếp tục phân tích các kỹ thuật mã hóa được ám chỉ trong phần chuỗi của PeStudio có thể được thực hiện bằng cách phân tích các chuỗi ngẫu nhiên và các hàm với địa chỉ bộ nhớ tương ứng của chúng.

Lấy ví dụ về chuỗi và hàm sau đây:

```
MarkMokeOsesShwaSkegpornlimemim  
KERNEL32.GetStartupInfo
```

Chúng ta có thể thấy:

- Khi thực hiện tìm kiếm chuỗi MarkMokeOsesShwaSkegpornlimemim, chúng ta được đưa đến địa chỉ bộ nhớ 0x00433dc1.
- Khi thực hiện tìm kiếm hàm KERNEL32.GetStartupInfo, chúng ta được đưa đến địa chỉ bộ nhớ 0x00433de1.

Lưu ý sự gần nhau trong địa chỉ bộ nhớ của hai mục này:

|            |       |                                   |
|------------|-------|-----------------------------------|
| 0x00433dc1 | dec   | ebp                               |
| 0x00433dc2 | popal |                                   |
| 0x00433dc3 | jb    | 0x433e30                          |
| 0x00433dc5 | dec   | ebp                               |
| 0x00433dc6 | outsd | dx, dword [esi]                   |
| 0x00433dc7 | imul  | esp, dword [ebp + 0x4f], 0x73     |
| 0x00433dcb | jae   | 0x433e21                          |
| 0x00433dce | push  | 0x6b536177 ; 'waSk'               |
| 0x00433dd3 | jo    | 0x433e46                          |
| 0x00433dd7 | jb    | 0x433e47                          |
| 0x00433dd9 | insb  | byte es:[edi], dx                 |
| 0x00433dda | imul  | ebp, dword [ebp + 0x65], 0x6d696d |
| 0x00433de1 | dec   | ebx                               |

Tại đây, chúng ta có thể nhận thấy rằng chuỗi và hàm gần gũi trong mã đã được giải mã, chỉ ra một mối quan hệ giữa hai mục này. Việc đặt cạnh nhau các chuỗi ngẫu nhiên với các lệnh gọi hệ thống có thể cho thấy một nỗ lực mã hóa hoặc có thể là một phần của một cơ chế phức tạp hơn, chẳng hạn như một quy trình mã hóa/giải mã tùy chỉnh sử dụng những chuỗi này.

Một điểm thú vị khác có thể được tìm thấy giữa hai địa chỉ bộ nhớ này:

|            |       |                                   |
|------------|-------|-----------------------------------|
| 0x00433dc1 | dec   | ebp                               |
| 0x00433dc2 | popal |                                   |
| 0x00433dc3 | jb    | 0x433e30                          |
| 0x00433dc5 | dec   | ebp                               |
| 0x00433dc6 | outsd | dx, dword [esi]                   |
| 0x00433dc7 | imul  | esp, dword [ebp + 0x4f], 0x73     |
| 0x00433dcb | jae   | 0x433e21                          |
| 0x00433dce | push  | 0x6b536177 ; 'waSk'               |
| 0x00433dd3 | jo    | 0x433e46                          |
| 0x00433dd7 | jb    | 0x433e47                          |
| 0x00433dd9 | insb  | byte es:[edi], dx                 |
| 0x00433dda | imul  | ebp, dword [ebp + 0x65], 0x6d696d |
| 0x00433de1 | dec   | ebx                               |



Một lệnh có tham chiếu đến `waSk`, đây là một phần nhỏ của chuỗi đã đề cập trước đó. Mẫu này có thể thấy lại với việc sử dụng các chuỗi có vẻ ngẫu nhiên và một lệnh gọi hàm.

Ta có một ví dụ khác:

- String: AsksmaceaglyBubuPulsKaifTeasMistPeelGhisPrimChaoLyrreroeno
  - + Memory Address: 0x004337a2
- Function: KERNEL32.MulDiv
  - + Memory Address: 0x004337dc
- String portion: isPr
  - + Memory Address: 0x004337c7

|            |       |                                     |
|------------|-------|-------------------------------------|
| 0x004337a2 | inc   | ecx                                 |
| 0x004337a3 | jae   | 0x433810                            |
| 0x004337a5 | jae   | 0x433814                            |
| 0x004337a7 | popal |                                     |
| 0x004337a8 | arpl  | word [ebp + 0x61], sp               |
| 0x004337ab | insb  | byte es:[di], dx                    |
| 0x004337ad | jns   | 0x4337f1                            |
| 0x004337af | jne   | 0x433813                            |
| 0x004337b1 | jne   | 0x433803                            |
| 0x004337b3 | jne   | 0x433821                            |
| 0x004337b5 | jae   | 0x433802                            |
| 0x004337b7 | popal |                                     |
| 0x004337b8 | imul  | esp, dword [esi + 0x54], 0x4d736165 |
| 0x004337bf | imul  | esi, dword [ebx + 0x74], 0x6c656550 |
| 0x004337c6 | inc   | edi                                 |
| 0x004337c7 | push  | 0x72507369 ; 'isPr'                 |
| 0x004337cc | imul  | ebp, dword [ebp + 0x43], 0x4c6f6168 |
| 0x004337d3 | jns   | 0x433847                            |
| 0x004337d5 | jb    | 0x433847                            |
| 0x004337d8 | outsb | dx, byte gs:[esi]                   |
| 0x004337da | outsd | dx, dword [esi]                     |
| 0x004337db | add   | byte [ebx + 0x45], cl               |
| 0x004337de | push  | edx                                 |
| 0x004337dc | dec   | ebx                                 |
| 0x004337dd | inc   | ebp                                 |
| 0x004337de | push  | edx                                 |
| 0x004337df | dec   | esi                                 |
| 0x004337e0 | inc   | ebp                                 |
| 0x004337e1 | dec   | esp                                 |

Lưu ý: Ta phải thêm địa chỉ hàm tương ứng. Cutter sẽ tự động ẩn nó ngay khi có bất kỳ sự di chuyển nào được thực hiện.

Một ví dụ khác:

- String: CellrotoCrudUntohighCols
  - + Memory Address: 0x0043396c
- Function: KERNEL32.CreateFile
  - + Memory Address: 0x00433985
- String portion: ighC
  - + Memory Address: 0x0043397c

|            |       |                     |
|------------|-------|---------------------|
| 0x0043396c | inc   | ebx                 |
| 0x0043396d | insb  | byte es:[edi], dx   |
| 0x0043396f | insb  | byte es:[edi], dx   |
| 0x00433970 | jb    | 0x4339e1            |
| 0x00433972 | je    | 0x4339e3            |
| 0x00433974 | inc   | ebx                 |
| 0x00433975 | jb    | 0x4339ec            |
| 0x00433977 | push  | ebp                 |
| 0x00433979 | outsb | dx, byte [esi]      |
| 0x0043397a | je    | 0x4339eb            |
| 0x0043397c | push  | 0x43686769 ; 'ighC' |
| 0x00433981 | outsd | dx, dword [esi]     |
| 0x00433982 | insb  | byte es:[edi], dx   |
| 0x00433983 | jae   | 0x433985            |
| 0x00433985 | dec   | ebx                 |

### 3.9. Tổng kết

#### 3.9.1. Nhận xét

Trên đây là các bước phân tích tĩnh. Trong khi phân tích tĩnh đóng vai trò là bước nền tảng trong việc hiểu các đặc điểm của mẫu phần mềm độc hại đã được phân tích, việc bổ sung điều này bằng các kỹ thuật phân tích động và hành vi là rất quan trọng để có cái nhìn sâu sắc về hành vi thực tế và tác động của nó trên hệ thống. Hơn nữa, việc áp dụng các phương pháp phân loại dựa trên YARA cung cấp thêm một lớp phân tích khác, cho phép xác định các mẫu hoặc chữ ký phần mềm độc hại đã biết. Việc tích hợp các nền tảng phân tích lại sẽ cung cấp thêm ngữ cảnh và xác thực, góp phần vào việc hiểu biết toàn diện hơn về mẫu đã được phân tích.

#### 3.9.2. Kết luận

Sử dụng tất cả các thông tin đã thu thập, chúng ta có thể rút ra kết luận sau:

- Tập invoice\_2318362983713\_823931342io.pdf.exe đã bị nhiễm dịch vụ phát hiện phần mềm độc hại đánh dấu, xác nhận rằng nó thực sự độc hại.

- Tập này chứa một URL nhưng là corect.com, tuy nhiên, URL này không tiết lộ thông tin thú vị nào.
- Xem xét địa chỉ thô và địa chỉ ảo, chúng ta nhận thấy rằng tệp .text bên trong invoice\_2318362983713\_823931342io.pdf.exe có địa chỉ ảo nhỏ hơn so với địa chỉ thô. Các tác giả phần mềm độc hại thường "nén" tệp bằng cách bao gồm mã không cần thiết để mã hóa mã độc thực sự.
- Xem xét các cuộc gọi API, chúng ta có thể nhận thấy một số cuộc gọi đáng ngờ:
  - + GetCapture
  - + GetWindowTextLength
  - + GetEnvironmentVariable
  - + GetLogicalDrives
  - + GetDriveType
  - + WriteFile
  - + FindFirstFile
  - + GetClipboardOwner
  - + GetClipboardData
- Khi điều tra các cuộc gọi API và chức năng của chúng, cùng với các cuộc gọi cụ thể được sử dụng cùng nhau, chúng ta có thể chỉ ra rằng phần mềm độc hại đang cố gắng thu thập dữ liệu của người dùng khi họ thực hiện các hành động nhất định. Phần mềm độc hại cũng đang tạo ra các tệp trong hệ thống tệp. Nó có khả năng đang cố gắng thu thập dữ liệu nhạy cảm như mật khẩu, thông tin ngân hàng và bất kỳ thông tin cá nhân nào khác. Chúng ta có thể suy luận rằng có thể đang xảy ra keylogging, một phương pháp theo dõi các phím gõ trên máy tính. Ngoài ra, phần mềm độc hại cũng đang truy xuất dữ liệu máy tính, chẳng hạn như loại ổ đĩa và ổ đĩa.
- Xem xét các cuộc gọi chức năng khả nghi khác cùng với các thư viện mà nó đang truy cập, chúng ta có thể suy đoán rằng phần mềm độc hại đang mã hóa các cuộc gọi chức năng bằng cách tạo ra một chuỗi văn bản ngẫu nhiên, sau đó là một thư viện được truy cập. Ví dụ: BagsSpicDollBikeAzonPoopHamsPyasmap theo sau là

KERNEL32.SetCurrentDirectory. Cuộc gọi chức năng khả nghi được mã hóa với BagsSpicDollBikeAzonPoopHamsPyasmap nhưng sau đó được theo sau bởi KERNEL32.SetCurrentDirectory, điều này càng củng cố giả thuyết của chúng ta về việc mã hóa đang diễn ra.

- Đáng chú ý là các thư viện SHLWAPI.dll, KERNEL32.dll, và USER32.dll đang được truy cập, càng chỉ ra hoạt động độc hại.
- Trong việc sử dụng Capa, nó đã tiết lộ khả năng của phần mềm độc hại mẫu. Capa đã liên kết phần mềm độc hại với TACTIC ATT&CK: DEFENSE EVASION -> Virtualization/Sandbox Evasion, cho thấy phần mềm độc hại đang cố gắng tránh bị phân tích bởi các nhà phân tích phần mềm độc hại. Nó có thể đang cố gắng sử dụng một quy trình để tự hủy nếu phát hiện đang ở trong một máy ảo.
- Capa cũng đã liên kết phần mềm độc hại với OBJECTIVE MBC: ANTI-BEHAVIORAL ANALYSIS -> Virtual Machine Detection, điều này càng cho thấy phần mềm độc hại đang cố gắng tránh phân tích hành vi bằng cách sử dụng phát hiện máy ảo.
- Sử dụng Cutter để tạo ra một đồ thị dựa trên các cuộc gọi API. Dựa vào thông tin trước đó, chúng ta có thể suy luận rằng các cuộc gọi API đang diễn ra theo chuỗi. Điều này hữu ích trong việc tiết lộ cho chúng ta cách mà phần mềm độc hại đang cố gắng truy cập dữ liệu. Nó cho thấy hành vi động của nó theo thứ tự xảy ra.
- Cutter cũng tiết lộ một cuộc gọi API cụ thể: IsBadReadPtr. Hàm này xác minh rằng quy trình gọi có quyền truy cập đọc vào một vùng bộ nhớ xác định. Điều này càng cho thấy nỗ lực của phần mềm độc hại trong việc thu thập dữ liệu từ một cơ sở dữ liệu nhạy cảm.